

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2019

Huấn luyện viên: Zhang Ruizhe

Tháng 5 năm 2019

Nguồn gốc: Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2019

IOI China candidate team papers / National training team 2019 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2019. Tập được tạo bằng XeTeX và có lớp văn bản đọc được; tuy nhiên các trang có watermark chéo, nên kết quả trích xuất xen thêm chữ từ watermark vào nội dung chính. Bản LaTeX này chuyển ngữ phần nhận diện tập sách, mục lục và ba bài viết đầu tiên: “Tính chất và ứng dụng của hai loại dãy truy hồi” của Zhong Ziqian và “Bàn về bài toán bước đi ngẫu nhiên trên mô hình đồ thị” của Wang Xiuhuan, và “Báo cáo ra đề *Bài toán nhỏ dễ*” của Yang Junzhao.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn ứng viên đội tuyển quốc gia IOI Trung Quốc năm 2019*. Trang bìa ghi huấn luyện viên là Zhang Ruizhe và thời điểm phát hành là tháng 5 năm 2019. Tập PDF gồm 231 trang.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả
1	Tính chất và ứng dụng của hai loại dãy truy hồi	Zhong Ziqian
17	Bàn về bài toán bước đi ngẫu nhiên trên mô hình đồ thị	Wang Xiuhuan
27	Báo cáo ra đề <i>Bài toán nhỏ dễ</i>	Yang Junzhao
38	Bàn về bài toán tô màu đỉnh của đồ thị	Gao Jiaxuan
55	Bàn về các bài toán liên quan tới đếm đường đi trên lưới	Dai Yan
69	Mở rộng một số bài toán số học trong thi thuật toán và bước đầu về số nguyên Gauss	Li Jiaheng
89	Báo cáo ra đề <i>Bài luyện cơ bản về cây tròn-vuông</i>	Fan Zhiyuan
104	Báo cáo ra đề <i>Đếm điểm nguyên</i> và một số nghiên cứu về số nguyên Gauss	Xu Yixuan
122	Bàn về thuật toán chia để trị trên cây	Zhang Zheyu
130	Báo cáo ra đề <i>Tổng tổ hợp</i>	Wu Siyang
141	Bàn về một lớp cấu trúc dữ liệu ngắn gọn	Wang Siqi
152	Thuật toán liên quan tới truy vấn chu kỳ sâu con và ứng dụng	Chen Sunli
165	Báo cáo ra đề <i>Công viên</i>	Wu Zuotong
192	Bàn về cấu trúc dữ liệu có thể truy vết lịch sử	Kong Chaozhe
202	Bàn về ứng dụng của ma trận Young trong thi tin học	Yuan Fangzhou

Tính chất và ứng dụng của hai loại dãy truy hồi

Zhong Ziqian, Trường Trung học số 3 Phúc Châu

Tóm tắt

Dãy truy hồi tuyến tính và dãy truy hồi đa thức là hai loại dãy truy hồi thường gặp trong toán học. Bài viết này giới thiệu định nghĩa, tính chất và thuật toán liên quan của hai loại dãy ấy, đồng thời trình bày một số ứng dụng của chúng trong thi tin học.

Lời nói đầu

Dãy truy hồi tuyến tính đã được đưa vào giới thi thuật toán ít nhất năm năm, nhưng vẫn chưa được phổ biến thật rộng rãi. Dãy truy hồi đa thức là một mở rộng tự nhiên của dãy truy hồi tuyến tính và chỉ mới được đưa vào thi tin học trong khoảng hai năm gần đây. Bài viết này mong muốn giới thiệu có hệ thống các tính chất và công dụng của hai loại dãy này trong thi tin học, để người đọc có đường hướng khi suy nghĩ về các bài toán liên quan.

Bài viết trước hết giới thiệu dãy truy hồi tuyến tính ở mục 1, rồi giới thiệu dãy truy hồi đa thức ở mục 2. Với cả hai loại dãy, bài viết trình bày định nghĩa, tính chất, thuật toán liên quan và bài ví dụ thực tế; riêng với dãy truy hồi tuyến tính còn có thêm một số ứng dụng liên quan tới đại số tuyến tính.

1. Dãy truy hồi tuyến tính

1.1. Định nghĩa

Định nghĩa 1.1. Ta gọi dãy có độ dài hữu hạn là dãy hữu hạn, và dãy có độ dài vô hạn là dãy vô hạn.

Định nghĩa 1.2. Ta gọi bậc của hạng tử cao nhất trong chuỗi lũy thừa hình thức F là bậc của F , ký hiệu $\deg(F)$, có thể bằng ∞ . Riêng đa thức không được quy ước có bậc là âm vô cùng $(-\infty)$.

Định nghĩa 1.3. Với dãy hữu hạn $\{a_0, a_1, a_2, \dots, a_{n-1}\}$, ta định nghĩa hàm sinh của nó là đa thức

$$A(x) = \sum_{i=0}^{n-1} a_i x^i.$$

Với dãy vô hạn $\{a_0, a_1, a_2, \dots\}$, ta định nghĩa tương tự hàm sinh của nó là chuỗi lũy thừa hình thức

$$A(x) = \sum_{i=0}^{\infty} a_i x^i.$$

Định nghĩa 1.4. Với dãy vô hạn $\{a_0, a_1, a_2, \dots\}$ và dãy hữu hạn không rỗng $\{r_0, r_1, \dots, r_{m-1}\}$, nếu với mọi $p \geq m-1$ ta có

$$\sum_{k=0}^{m-1} a_{p-k} r_k = 0,$$

thì gọi r là một công thức truy hồi tuyến tính của a . Nếu $r_0 = 1$, ta gọi r là công thức truy hồi tuyến tính của a . Dãy vô hạn có tồn tại công thức truy hồi tuyến tính được gọi là dãy truy hồi tuyến tính.

Với dãy hữu hạn $\{a_0, a_1, \dots, a_{n-1}\}$ định nghĩa tương tự, chỉ yêu cầu đẳng thức trên với $m-1 \leq p \leq n-1$. Bậc của công thức truy hồi tuyến tính là độ dài của nó trừ một. Công thức truy hồi tuyến tính của a có bậc nhỏ nhất được gọi là công thức truy hồi tuyến tính ngắn nhất của a .

1.2. Tính chất cơ bản và phương pháp nhận biết

Định lý 1.1. Với dãy vô hạn a và dãy hữu hạn không rỗng r như trên, gọi A và R là hàm sinh tương ứng của chúng. Khi đó r là công thức truy hồi tuyến tính của a khi và chỉ khi tồn tại đa thức S có bậc không vượt quá $m - 2$ sao cho

$$AR + S = 0.$$

Với dãy hữu hạn $\{a_0, \dots, a_{n-1}\}$, điều kiện tương ứng là

$$AR + S \equiv 0 \pmod{x^n}.$$

Chứng minh. Ta chứng minh trường hợp dãy vô hạn; trường hợp dãy hữu hạn tương tự. Với $k \geq m - 1$, hệ số của x^k trong $A(x)R(x)$ là

$$[x^k](A(x)R(x)) = \sum_{j=0}^{m-1} r_j a_{k-j} = 0.$$

Chọn S thích hợp để triệt tiêu các hệ số bậc thấp là đủ. $\square$

Hệ quả 1.1. Với dãy vô hạn a , gọi A là hàm sinh của nó. Khi đó a là dãy truy hồi tuyến tính khi và chỉ khi tồn tại một đa thức R có hệ số tự do bằng 1 và một đa thức S sao cho

$$A = \frac{S}{R}.$$

Bậc của công thức truy hồi tuyến tính ngắn nhất của a là giá trị nhỏ nhất có thể của $\max(\deg(R), \deg(S) + 1)$. Chứng minh là chuyển về trực tiếp từ định lý 1.1.

Định lý 1.2. Với ma trận $n \times n$ là M , dãy $\{I, M, M^2, M^3, \dots\}$ là dãy truy hồi tuyến tính, và bậc của công thức truy hồi tuyến tính ngắn nhất không vượt quá n .

Chứng minh. Gọi p là đa thức đặc trưng của M . Ta có $\deg(p) = n$, $[x^n]p(x) = 1$, và theo định lý Cayley–Hamilton, $p(M) = 0$. Viết $p(x) = \sum_{i=0}^n c_{n-i}x^i$. Từ

$$\sum_{i=0}^n c_{n-i}M^i = 0$$

và nhân thêm M^j , suy ra

$$\sum_{i=0}^n c_i M^{j+n-i} = 0.$$

Vậy $\{c_0, c_1, \dots, c_n\}$ là một công thức truy hồi tuyến tính bậc n của dãy ma trận này. $\square$

Định lý 1.3. Với các dãy truy hồi tuyến tính $a = \{a_i\}$, $b = \{b_i\}$, dãy hữu hạn $\{t_0, \dots, t_{m-1}\}$ và hằng số p , các dãy sau đều là dãy truy hồi tuyến tính:

$$\{t_0, \dots, t_{m-1}, a_0, a_1, \dots\}, \quad \{a_i p^i\}, \quad \{a_{i+1}\}, \quad \{a_i + b_i\},$$

$$\left\{ \sum_{j=0}^i a_j b_{i-j} \right\}, \quad \{a_i b_i\}.$$

Các chứng minh khá đơn giản, dùng hệ quả 1.1 và định lý 1.2 là đủ; do giới hạn độ dài nên lược bỏ.

1.3. Thuật toán liên quan

1.3.1. Tìm công thức truy đầy tuyến tính ngắn nhất

Trước hết xét dãy hữu hạn và giả sử mọi phép toán nằm trên một trường. Cách đơn giản là khử Gauss, với dãy dài n tốn $O(n^3)$. Thuật toán Berlekamp–Massey giảm thời gian xuống $O(n^2)$.

Với dãy $\{a_0, \dots, a_{n-1}\}$, Berlekamp–Massey tìm công thức truy đầy tuyến tính ngắn nhất $r^{(i)}$ cho mỗi tiền tố $\{a_0, \dots, a_i\}$. Đặt $|r^{(i)}| - 1 = l_i$. Rõ ràng $r^{(-1)} = \{1\}$ và $l_{i-1} \leq l_i$.

Bổ đề 1.1. Nếu $r^{(i-1)}$ không phải công thức truy đầy tuyến tính ngắn nhất của $\{a_0, \dots, a_i\}$, thì

$$l_i \geq \max(l_{i-1}, i + 1 - l_{i-1}).$$

Chứng minh. Phản chứng, giả sử $l_i \leq i - l_{i-1}$. Đặt

$$r^{(i-1)} = \{p_0, p_1, \dots, p_{l_{i-1}}\}, \quad r^{(i)} = \{q_0, q_1, \dots, q_{l_i}\}.$$

Theo định nghĩa của $r^{(i)}$, với $l_i \leq j \leq i$,

$$a_j = - \sum_{t=1}^{l_i} q_t a_{j-t}.$$

Do đó

$$\begin{aligned} - \sum_{j=1}^{l_{i-1}} p_j a_{i-j} &= \sum_{j=1}^{l_{i-1}} p_j \sum_{k=1}^{l_i} q_k a_{i-j-k} \\ &= \sum_{k=1}^{l_i} q_k \sum_{j=1}^{l_{i-1}} p_j a_{i-j-k} = - \sum_{k=1}^{l_i} q_k a_{i-k} = a_i. \end{aligned}$$

Vậy $r^{(i-1)}$ vẫn là công thức truy đầy cho tiền tố dài hơn, mâu thuẫn. $\square$

Dấu bằng trong bổ đề có thể đạt được. Gọi A là hàm sinh của a , còn $R^{(i)}$ là hàm sinh của $r^{(i)}$. Theo định lý 1.1 tồn tại $S^{(i)}$ sao cho

$$AR^{(i)} \equiv S^{(i)} \pmod{x^{i+1}}, \quad \deg(S^{(i)}) \leq l_i - 1.$$

Nếu $AR^{(i-1)} \equiv S^{(i-1)} \pmod{x^{i+1}}$, đặt $R^{(i)} = R^{(i-1)}$. Ngược lại

$$AR^{(i-1)} \equiv S^{(i-1)} + dx^i \pmod{x^{i+1}}.$$

Xét lần gần nhất công thức tăng bậc: tồn tại $p < i$ và c sao cho

$$AR^{(p-1)} \equiv S^{(p-1)} + cx^p \pmod{x^{p+1}}.$$

Nhân với $x^{i-p}dc^{-1}$ rồi trừ hai đồng dư, ta đặt được

$$R^{(i)} = R^{(i-1)} - x^{i-p}dc^{-1}R^{(p-1)}.$$

Khi đó $l_i = \max(l_{i-1}, i + 1 - l_{i-1})$. Chỉ cần duyệt i tăng dần, tính $R^{(i)}$ và cập nhật p, c khi $l_i > l_{i-1}$. Độ phức tạp là $O(n^2)$.

Định lý 1.4. Nếu dãy truy hồi tuyến tính a có công thức truy đầy tuyến tính ngắn nhất bậc không vượt quá s , thì công thức truy đầy tuyến tính ngắn nhất của tiền tố $\{a_0, a_1, \dots, a_{2s-1}\}$ chính là công thức truy đầy tuyến tính ngắn nhất của a . Chứng minh dùng bổ đề 1.1: nếu lần đầu thất bại ở vị trí $t \geq 2s$, bậc mới ít nhất $t + 1 - s > s$, mâu thuẫn. Vì vậy chỉ cần biết chặn trên s , tính $2s$ hạng đầu và chạy Berlekamp–Massey.

1.3.2. Tính một hạng của dãy truy hồi tuyến tính

Giả sử a thỏa công thức $\{r_0, \dots, r_{m-1}\}$, cần tính a_k . Đặt

$$G(F) = \sum_{i=0}^{\infty} a_i [x^i] F(x), \quad S(x) = \sum_{i=0}^{m-1} x^{m-1-i} r_i.$$

Từ công thức truy hồi, $G(S(x)T(x)) = 0$ với mọi đa thức T . Chia

$$x^k = S(x)U(x) + R(x), \quad \deg R < \deg S,$$

ta được

$$G(x^k) = G(x^k \bmod S(x)).$$

Dùng lũy thừa nhanh để tính $x^k \bmod S(x)$, rồi lấy tổ hợp tuyến tính các hạng đầu của a . Nếu phép lấy dư hai đa thức bậc $O(m)$ làm trong $O(m \log m)$, tổng thời gian là $O(m \log m \log k)$ [4].

1.4. Ứng dụng thường gặp

Nhờ định lý 1.2, dãy truy hồi tuyến tính thường gặp trong đại số tuyến tính. Dưới đây giả sử tính toán modulo một số nguyên tố lớn p .

1.4.1. Dãy vectơ và dãy ma trận

Với dãy vectơ hàng n chiều $\{t_0, t_1, \dots\}$, lấy ngẫu nhiên vectơ cột v rồi xét dãy vô hướng $\{t_0 v, t_1 v, \dots\}$. Theo bổ đề Schwartz–Zippel [5], với xác suất ít nhất $1 - n/p$, công thức truy đầy ngắn nhất của dãy vô hướng trùng với công thức của dãy vectơ.

Với dãy ma trận $n \times m$, lấy ngẫu nhiên vectơ hàng u và vectơ cột v , rồi xét $\{ut_0 v, ut_1 v, ut_2 v, \dots\}$. Xác suất thành công ít nhất $1 - (n + m)/p$.

1.4.2. Đa thức tối thiểu của ma trận

Đa thức tối thiểu của M là đa thức khác không bậc nhỏ nhất f sao cho $f(M) = 0$. Nếu $\{r_0, \dots, r_m\}$ là công thức truy đầy của $\{I, M, M^2, \dots\}$, thì

$$\sum_{i=0}^m r_{m-i} M^i = 0, \quad f(x) = \sum_{i=0}^m r_{m-i} x^i$$

là một đa thức triệt tiêu. Vậy đa thức tối thiểu tương ứng với công thức truy đầy tuyến tính ngắn nhất của dãy ma trận này, có bậc không vượt quá n .

Cụ thể, với u, v ngẫu nhiên, tính $\{uv, uMv, uM^2v, \dots, uM^{2n}v\}$. Vì $uM^{i+1} = (uM^i)M$, ma trận đặc tồn $O(n^3)$. Nếu M thưa có e vị trí khác không, mỗi bước tốn $O(n + e)$, tổng thời gian $O(n(n + e))$.

1.4.3. Tối ưu quy hoạch động

Xét quy hoạch động

$$f(i, j) = \sum_{t=0}^{m-1} f(i-1, t) c(t, j) \quad (i \geq 1, j \in [0, m)).$$

Gọi $F(i)$ là vectơ hàng các giá trị $f(i, j)$, và C là ma trận của c . Khi đó $F(i) = F(i-1)C$, nên $F(n) = F(0)C^n$. Thay vì lũy thừa ma trận $O(m^3 \log n)$, dùng định lý 1.2 để suy ra $\{F(n)\}$ là dãy truy hồi tuyến tính, tìm công thức truy đầy từ các hạng đầu rồi dùng mục 1.3.2. Độ phức tạp được nêu là

$$O(m^3 + m \log m \log n).$$

1.4.4. Hệ tuyến tính thưa

Cho ma trận đầy hạng A và vectơ hàng b , cần tìm x sao cho $Ax = b$. Vì $x = A^{-1}b$, xét công thức truy đầy ngắn nhất $\{r_0, \dots, r_{m-1}\}$ của $\{b, Ab, A^2b, \dots\}$. Ta có

$$\sum_{i=0}^{m-1} A^i b r_{m-1-i} = 0, \quad r_{m-1} \neq 0.$$

Nhân với A^{-1} , suy ra

$$A^{-1}b = -\frac{1}{r_{m-1}} \sum_{i=0}^{m-2} A^i b r_{m-2-i}.$$

Nếu A có e vị trí khác không, tính dãy $\{b, Ab, \dots, A^{2n}b\}$ tốn $O(ne)$, tổng thời gian $O(n(n+e))$.

1.4.5. Định thức ma trận thưa

Để tính $\det(A)$, tìm đa thức tối thiểu của ma trận thưa. Nếu mỗi giá trị riêng có bội hình học bằng một, đa thức tối thiểu là đa thức đặc trưng; hệ số tự do nhân $(-1)^n$ là định thức. Để tăng xác suất thỏa điều kiện này, nhân A với ma trận đường chéo ngẫu nhiên B . Khi đó AB thỏa với xác suất ít nhất $1 - (2n^2 - n)/p$ [7]. Tính $\det(AB)$, rồi chia cho $\det(B)$, là tích các phần tử đường chéo. Độ phức tạp $O(n(n+e))$.

1.4.6. Hạng ma trận thưa

Với ma trận $n \times m$ A , hạng của ma trận vuông bằng cấp trừ bội đại số của giá trị riêng 0. Nếu đa thức đặc trưng bằng đa thức tối thiểu nhân một lũy thừa của x , chỉ cần tìm đa thức tối thiểu rồi bỏ các thừa số x .

Sinh ngẫu nhiên ma trận đường chéo D_1 cỡ $m \times m$. Ma trận AD_1A^T đối xứng và có cùng hạng với A với xác suất ít nhất $1 - n^2/p$. Sinh tiếp ma trận đường chéo D_2 ; ma trận $D_2AD_1A^TD_2$ thỏa điều kiện đặc trưng/tối thiểu với xác suất ít nhất $1 - 4n^2/p$ [7]. Vì các ma trận đường chéo và A, A^T đều thưa, thời gian vẫn là $O(n(n+e))$.

1.5. Bài ví dụ

Ví dụ 1. Gapless Filling with Tetrominoes. Nguồn: Bytedance Camp 2019 Day3 Division A, Problem G, có chỉnh sửa.

Cho lưới $n \times 4$, cần dùng n tetromino phủ kín, mỗi ô đúng một lần. In số phương án modulo số nguyên tố p , với

$$1 \leq n \leq 10^9, \quad 2 \leq p \leq 10^9 + 9.$$

Dùng quy hoạch động nén trạng thái: $f[t][S]$ là số cách sau khi xét t hàng, trong đó S là tập ô chưa phủ ở ba hàng cuối. Đáp án là $f[n][\emptyset]$. Lời giải chính thức tìm các trạng thái đạt được rồi chuyển bằng ma trận. Theo định lý 1.2, $\{f[u][\emptyset]\}_{u=0}^\infty$ là dãy truy hồi tuyến tính; tìm công thức truy đầy bằng mục 1.3.1 rồi nhảy hạng bằng mục 1.3.2. Thử nghiệm thực tế tìm được công thức bậc 34.

Ví dụ 2. Expected Value. Nguồn: Ildar Gainullin Contest 1, Problem E, có giản lược.

Cho đồ thị phẳng đơn, vô hướng, liên thông. Quân cờ bắt đầu ở đỉnh 1, mỗi bước chọn đều một cạnh kề và đi theo cạnh đó. Tính kỳ vọng số bước để tới đỉnh n , modulo số nguyên tố p , với

$$1 \leq n \leq 2000, \quad 10^9 < p < 1.01 \times 10^9.$$

Đồ thị phẳng có $m \leq 3n - 6$ cạnh [8]. Gọi f_x là kỳ vọng từ đỉnh x , d_x là bậc của x . Khi đó

$$f_x = \begin{cases} 1 + \sum_{(x,y) \in E} \frac{f_y}{d_x}, & x \neq n, \\ 0, & x = n. \end{cases}$$

Mà trận hệ số chỉ có $O(n + m)$ vị trí khác không, nên dùng mục 1.4.4, độ phức tạp $O(n^2)$. Một cách khác xem ở [9].

2. Dãy truy hồi đa thức

Thuật ngữ tiếng Anh thường dùng cho loại dãy này là *P-recursive sequences*.

2.1. Định nghĩa

Ta dùng lại các định nghĩa dãy hữu hạn, dãy vô hạn, bậc chuỗi lũy thừa hình thức và hàm sinh. Giả sử tính toán trên trường $\mathbb{K}$.

Định nghĩa 2.1. Với dãy vô hạn $a = \{a_i\}$ và dãy đa thức hữu hạn không rỗng $\{P_0, \dots, P_{m-1}\}$, nếu $P_0 \neq 0$ và với mọi $p \geq m - 1$

$$\sum_{k=0}^{m-1} a_{p-k} P_k(p) = 0,$$

thì gọi dãy P là công thức truy hồi đa thức của a . Dãy có tồn tại công thức như vậy gọi là dãy truy hồi đa thức. Với dãy hữu hạn, yêu cầu đẳng thức cho $m - 1 \leq p \leq n - 1$. Bậc truy hồi của $\{r_0, \dots, r_{m-1}\}$ là $m - 1$, bậc đa thức là $\max_i \deg(r_i)$.

Định nghĩa 2.2. Chuỗi lũy thừa hình thức $A(x)$ gọi là vi phân hữu hạn nếu tồn tại đa thức

$$Q_0(x), \dots, Q_{m-1}(x),$$

với $Q_{m-1} \neq 0$, sao cho

$$\sum_{i=0}^{m-1} Q_i(x) A^{(i)}(x) = 0.$$

Chuỗi $A(x)$ gọi là đại số nếu nó đại số trên $\mathbb{K}(x)$, tức là nghiệm của một phương trình đa thức có hệ số trong trường phân thức hữu tỷ $\mathbb{K}(x)$. Thuật ngữ tiếng Anh của “vi phân hữu hạn” là *D-finite*.

2.2. Tính chất cơ bản và phương pháp nhận biết

Định lý 2.1. Nếu hàm sinh thường của $a = \{a_i\}$ là $A(x)$, thì a là dãy truy hồi đa thức khi và chỉ khi A vi phân hữu hạn.

Chứng minh. Chiều đủ: vì

$$x^j A^{(i)}(x) = \sum_{n=0}^{\infty} a_{n+i-j} x^n \prod_{t=1}^i (n+t-j)$$

(các hạng có chỉ số âm xem là 0), thay vào quan hệ vi phân rồi lấy hệ số x^n với n đủ lớn sẽ cho một công thức truy hồi đa thức.

Chiều cần: viết công thức truy hồi dưới dạng

$$\sum_{k=0}^{m-1} a_{p+k} P'_k(p) = 0 \quad (p \geq 0).$$

Mỗi $P'_i(n)$ có thể biểu diễn thành tổ hợp tuyến tính của $\prod_{t=0}^{j-1} (n+i-t)$. Mặt khác

$$x^i \sum_{n=0}^{\infty} a_{n+i} x^n \prod_{t=0}^{j-1} (n+i-t) = x^i A^{(j)}(x) - \sum_{n=0}^{i-1} a_n x^n \prod_{t=0}^{j-1} (n-t).$$

Gom hạng sẽ được

$$\sum_i Q'_i(x) A^{(i)}(x) = S(x)$$

với S là đa thức. Nếu $S \neq 0$, lấy đạo hàm $\deg(S) + 1$ lần ở hai vế. $\square$

Bổ đề 2.1. Chuỗi $u(x)$ vi phân hữu hạn khi và chỉ khi

$$\dim_{\mathbb{K}(x)} \text{span}_{\mathbb{K}(x)} \{u(x), u'(x), \dots\}$$

hữu hạn. Một chiều lấy quan hệ phụ thuộc tuyến tính giữa $u, u', \dots$; chiều còn lại dùng quan hệ vi phân để biểu diễn mọi đạo hàm bậc cao bằng hữu hạn đạo hàm đầu. $\square$

Bổ đề 2.2. Nếu u đại số theo x , thì u' cũng đại số. Thật vậy, với đa thức tối thiểu $P(x, u) = 0$, lấy đạo hàm:

$$0 = \frac{\partial P(x, y)}{\partial x} \Big|_{y=u} + u' \frac{\partial P(x, y)}{\partial y} \Big|_{y=u},$$

nên

$$u' = - \frac{\frac{\partial P(x, y)}{\partial x} \Big|_{y=u}}{\frac{\partial P(x, y)}{\partial y} \Big|_{y=u}}.$$

Định lý 2.2. Nếu u vi phân hữu hạn, v đại số, và $u(v(x))$ là một chuỗi lũy thừa hình thức, thì $u(v(x))$ vi phân hữu hạn.

Chứng minh. Đặt $y = u(v(x))$. Từ quy tắc dây chuyền, mọi $y^{(i)}$ là tổ hợp tuyến tính của

$$u(v(x)), u'(v(x)), \dots$$

với hệ số trong $\mathbb{K}[v', v'', \dots]$. Theo bổ đề 2.2, các $v^{(i)}$ nằm trong $\mathbb{K}(x, v)$. Vì u vi phân hữu hạn, $\text{span}_{\mathbb{K}(x, v)} \{u(v), u'(v), \dots\}$ hữu hạn chiều; do v đại số, $[\mathbb{K}(x, v) : \mathbb{K}(x)]$ hữu hạn. Bởi bổ đề 2.1, y vi phân hữu hạn. $\square$

Định lý 2.3. Với các dãy truy hồi đa thức $a = \{a_i\}$, $b = \{b_i\}$, dãy hữu hạn $\{t_0, \dots, t_{m-1}\}$ và hằng số p , các dãy sau đều là dãy truy hồi đa thức:

$$\{t_0, \dots, t_{m-1}, a_0, a_1, \dots\}, \quad \{a_i p^i\}, \quad \{a_{i+1}\}, \quad \{a_i + b_i\}, \\ \left\{ \sum_{j=0}^i a_j b_{i-j} \right\}, \quad \{a_i b_i\}.$$

Ba tính chất đầu theo định nghĩa. Với tổng, dùng $V_t = \text{span}_{\mathbb{K}(x)} \{t, t', t'', \dots\}$ và $V_{u+v} \subseteq V_u \oplus V_v$. Với tích chập, hàm sinh là uv , dùng quy tắc tích và ánh xạ $\varphi : V_u \otimes V_v \rightarrow \mathbb{K}((x))$, $\varphi(y \otimes z) = yz$. Tính chất cuối phức tạp hơn, xem định lý 6.4.12 trong [10].

2.3. Thuật toán liên quan

2.3.1. Tìm công thức truy hồi đa thức

Vì bậc đa thức chưa biết, thường liệt kê hoặc suy ra bậc s , rồi tìm công thức bậc truy hồi nhỏ nhất dưới bậc đó. Với công thức bậc $m \{P_0, \dots, P_m\}$,

$$\sum_{k=0}^m a_{p-k} P_k(p) = 0 \quad (m \leq p \leq n-1).$$

Đặt $Q_i(x) = P_i(x+m)$, ta có

$$\sum_{k=0}^m a_{p+k} Q_k(p) = 0 \quad (0 \leq p \leq n-m-1).$$

Giả sử bậc truy hồi không vượt quá m_0 , đặt $t_{k,u} = [x^u]Q_k(x)$; số biến là $(m_0+1)(s+1)$. Lấy $n \geq (s+2)(m_0+1)$ hạng đầu và lập $n-m_0$ phương trình. Ma trận không đầy hạng vì nhân mọi Q_i với một hằng số vẫn là nghiệm; ta khử theo thứ tự các biến, gán giá trị khác không cho biến tự do đầu tiên sao cho nhận được $Q_m \neq 0$. Sau đó dùng nhị thức để đổi ngược về P_i . Độ phức tạp $O(n^2ms)$. Với dữ liệu ngẫu nhiên và n đủ lớn, xác suất tìm đúng công thức là cao.

2.3.2. Tính một hạng của dãy truy hồi đa thức

Giả sử a thỏa công thức bậc $m \{P_0, \dots, P_m\}$, biết $a_0, \dots, a_{m-1}$, và bậc đa thức là d . Nếu $P_0(x) \neq 0$ với mọi số nguyên $x \in [m, n]$, có thể truy đẩy trực tiếp:

$$a_p = -\frac{1}{P_0(p)} \sum_{j=1}^m a_{p-j} P_j(p),$$

tốn $O(nmd)$.

Khi $m, d \ll n$, tính trước $\prod_{i=m}^n P_0(i)a_n$, rồi chia cho $\prod_{i=m}^n P_0(i)$. Với $n' = n-m+1$, đặt

$$u_i = \left\{ a_j \prod_{t=0}^{i-1} P_0(t+m) \right\}_{j=i}^{i+m-1}.$$

Khi đó $u_{n'}$ chứa $\prod_{i=m}^n P_0(i)a_n$. Quan hệ $u_i M(i) = u_{i+1}$ được mô tả bởi ma trận

$$M(i) = \begin{bmatrix} 0 & 0 & \dots & 0 & 0 & -P_m(i+m) \\ P_0(i+m) & 0 & \dots & 0 & 0 & -P_{m-1}(i+m) \\ 0 & P_0(i+m) & \dots & 0 & 0 & -P_{m-2}(i+m) \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & \dots & P_0(i+m) & 0 & -P_2(i+m) \\ 0 & 0 & \dots & 0 & P_0(i+m) & -P_1(i+m) \end{bmatrix},$$

nên

$$u_{n'} = u_0 \prod_{i=0}^{n'-1} M(i).$$

Xem $M(x)$ là ma trận đa thức bậc không vượt quá d . Chọn ngưỡng S và tính

$$T_S(x) = \prod_{i=0}^{S-1} M(Sx+i).$$

Mỗi phần tử có bậc không vượt quá dS . Ta chỉ cần các giá trị điểm tại $x = 0, 1, \dots, \lfloor (n' - S)/S \rfloor$, rồi nhân các ma trận giá trị ấy và các hạng lẻ còn lại.

Để tính nhanh các giá trị điểm, đặt $T_w(x) = \prod_{i=0}^{w-1} M(x+i)$ và nhân đôi w , dùng

$$T_{2w}(x) = T_w(x)T_w(x+w).$$

Phép dịch giá trị điểm dùng nội suy Lagrange: nếu biết $h(0), \dots, h(d)$, thì

$$\begin{aligned} h(m+k) &= \sum_{i=0}^d h(i) \prod_{\substack{j=0 \\ j \neq i}}^d \frac{m+k-j}{i-j} \\ &= \left(\prod_{j=0}^d (m+k-j) \right) \left(\sum_{i=0}^d \frac{h(i)}{i!(d-i)!(-1)^{d-i}} \frac{1}{m+k-i} \right). \end{aligned}$$

Phần tích có thể lấy bằng tích tiền tố trên đoạn $[k-d, k+d]$, phần còn lại là tích chập, xử lý bằng FFT trong $O(d \log d)$.

Chọn S nhỏ nhất thỏa $dS \geq (n' - S)/S$, khi đó

$$S = O(\sqrt{n/d}), \quad T = O\left(\sqrt{nd}(m^3 + m^2 \log(nd))\right).$$

Nếu $P_0 \neq 1$, tích $\prod_{i=m}^n P_0(i)$ cũng tính bằng cùng phương pháp với dãy bậc 1, không đổi nút nghẽn.

2.4. Bài ví dụ

Ví dụ 3. Chef and Same Old Recurrence 2. Nguồn: CodeChef JADUGAR2, có chỉnh sửa.

Cho K, A, B , định nghĩa

$$\begin{cases} a_1 = K, \\ a_n = Aa_{n-1} + B \sum_{i=1}^{n-1} a_i a_{n-i}, \quad n > 1. \end{cases}$$

Cần tính $a_1, \dots, a_N$ modulo $10^9 + 7$, nhưng chỉ in xor của các giá trị, với $1 \leq N \leq 10^7, 1 \leq B, K < 10^9 + 7, 0 \leq A < 10^9 + 7$. Đặt $a_0 = 0$ và gọi g là hàm sinh. Từ truy hồi suy ra

$$g = Kx + Axg + Bg^2.$$

Vậy g đại số trên $\mathbb{K}(x)$, nên vi phân hữu hạn; a là dãy truy hồi đa thức. Dùng mục 2.3.1 để tìm công thức rồi truy đẩy, thời gian $O(N)$.

Ví dụ 4. Jump Jump Jump. Nguồn: Grand Prix of Zhejiang, Problem J, có chỉnh sửa.

Một con thỏ bắt đầu ở $(0, 0)$, mỗi bước chọn đều một trong k vectơ (dx_i, dy_i) khác nhau rồi nhảy theo vectơ ấy. Tại mỗi $(x, x), x \geq 1$, có một bẫy. Cho n , với mỗi $x \in [1, n]$ hãy tính xác suất thỏ rơi vào bẫy (x, x) modulo 998244353. Các $dx_i, dy_i \in [0, 3]$ và không đồng thời bằng 0; $1 \leq n \leq 10^5, 1 \leq k \leq 15$.

Định lý 2.4. Nếu $F(s, t)$ là chuỗi lũy thừa hình thức hữu tỷ theo s, t , thì

$$\text{diag } F = \sum_n x^n [s^n t^n] F(s, t)$$

là chuỗi lũy thừa hình thức đại số theo x . Chứng minh xem [10, định lý 6.3.3].

Gọi p_t là xác suất trong mô hình không có bẫy rằng thỏ từng đi qua (t, t) . Đặt

$$G(s, t) = \frac{1}{k} \sum_{i=1}^k s^{dx_i} t^{dy_i}, \quad F(s, t) = \sum_{i=0}^{\infty} G(s, t)^i = \frac{1}{1 - G(s, t)}.$$

Khi đó $p_t = [x^t]$ diag F , nên theo định lý 2.4 và 2.2, p là dãy truy hồi đa thức. Tính một số hạng đầu bằng quy hoạch động, tìm công thức bằng mục 2.3.1, rồi truy đẩy được $p_1, \dots, p_n$.

Gọi h_t là xác suất rơi vào bẫy (t, t) . Khi đó

$$h_t = p_t - \sum_{s=1}^{t-1} h_s p_{t-s}.$$

Với

$$P = \sum_{i=1}^{\infty} x^i p_i, \quad H = \sum_{i=1}^{\infty} x^i h_i,$$

ta có $H = P - HP$, tức $H = P/(1 + P)$. Biết $P \bmod x^{n+1}$, dùng nghịch đảo đa thức để tính $H \bmod x^{n+1}$ trong $O(n \log n)$ [4].

Ví dụ 5. Đếm đơn giản. Nguồn: bài kiểm tra lẫn nhau của đội tuyển tập huấn năm 2019, có chỉnh sửa.

Tính số đồ thị có hướng không chu trình có nhãn trên n đỉnh, mỗi cạnh có một trong k màu, mỗi đỉnh có không quá một cạnh đi ra, và số cạnh đi vào mỗi đỉnh thuộc S . Hai đồ thị khác nhau nếu khác cạnh hoặc khác màu cạnh.

$$1 \leq n \leq 9 \times 10^8, \quad 1 \leq k \leq 10^7, \quad 0 \in S, \quad S \subseteq [0, 3].$$

Đối tượng ứng sang rừng có gốc có nhãn: mỗi đỉnh có số con thuộc S , mỗi cạnh có k màu. Gọi f_n là số cây hợp lệ, $g_{a,n}$ là số rừng gồm đúng a cây, h_n là số rừng. Dùng hàm sinh mũ $A(x) = \sum a_i x^i / i!$. Gọi các hàm sinh là $F(x), G_a(x), H(x)$. Từ tính chất hàm sinh mũ [12],

$$G_a(x) = \frac{F(x)^a}{a!}, \quad F(x) = \sum_{t \in S} x k^t G_t(x) = \sum_{t \in S} x k^t \frac{F(x)^t}{t!},$$

$$H(x) = \sum_{t=0}^{\infty} \frac{F(x)^t}{t!} = e^{F(x)}.$$

Đáp án là $n![x^n]H(x)$. Vì F đại số và e^x vi phân hữu hạn, định lý 2.2 cho $H = e^F$ vi phân hữu hạn. Theo định lý 2.3, $\{t![x^t]H(x)\}$ là dãy truy hồi đa thức. Dùng mục 2.3.1 để khử ra công thức, rồi mục 2.3.2 để tính đáp án.

3. Tổng kết

Dãy truy hồi tuyến tính và dãy truy hồi đa thức là hai loại dãy rất thường gặp, có ứng dụng rộng rãi trong toán học, nhưng hiện vẫn chưa phổ biến trong thi tin học. Bài viết đã giới thiệu định nghĩa, tính chất, thuật toán liên quan và ứng dụng của chúng, với hy vọng giúp người đọc có hiểu biết ban đầu và mong rằng sẽ có thêm nhiều bài toán thú vị thuộc lớp này xuất hiện trong thi tin học.

Trong lĩnh vực này vẫn còn nhiều vấn đề cần giải quyết. Do trình độ tác giả còn hạn chế, mong bài viết có thể đóng vai trò gợi mở và thu hút thêm nhiều độc giả nghiên cứu các bài toán liên quan.

Lời cảm ơn

Xin cảm ơn Hội Máy tính Trung Quốc đã cung cấp nền tảng học tập và trao đổi. Xin cảm ơn huấn luyện viên Zhang Ruizhe của đội tuyển tập huấn quốc gia đã hướng dẫn. Xin cảm ơn các thầy cô Huang Zhigang, Song Lilin, Huang Xiaoyan và Wei Lizhen đã bồi dưỡng và dạy dỗ tôi. Xin cảm ơn Du Yuhao, Mao Xiao, Wang Xiuhuan, Zhu Zhenting đã giúp đỡ bài viết này. Xin cảm ơn Kong Chaozhe, Liu Cheng'ao, Zeng Yaohui, Zhang Qingchuan, Wu Hang đã kiểm tra bản thảo. Xin cảm ơn cha mẹ đã thấu hiểu và ủng hộ tôi.

Tài liệu tham khảo

1. Mao Xiao. Một số nghiên cứu về công thức truy hồi của dãy số. Tập luận văn ứng viên đội tuyển quốc gia IOI Trung Quốc 2017, 2017.
2. Wikipedia contributors. Cayley–Hamilton theorem. Wikipedia, https://en.wikipedia.org/wiki/Cayley-Hamilton_theorem.
3. Massey J. Shift-register synthesis and BCH decoding. IEEE Transactions on Information Theory, 1969.
4. Peng Yuxiang. Introduction to Polynomials. WC2016, 2016.
5. Wikipedia contributors. Schwartz–Zippel lemma. Wikipedia, https://en.wikipedia.org/wiki/Schwartz-Zippel_lemma.
6. Wiedemann, Douglas. Solving sparse linear equations over finite fields. IEEE Transactions on Information Theory 32.1, 1986.
7. Chen, Li, et al. Efficient matrix preconditioners for black box linear algebra. Linear Algebra and its Applications 343, 2002.
8. Wikipedia contributors. Planar graph. Wikipedia, https://en.wikipedia.org/w/index.php?title=Planar_graph.
9. Wang Xiuhan. Bàn về bài toán bước đi ngẫu nhiên trên mô hình đồ thị. Tập luận văn ứng viên đội tuyển quốc gia IOI Trung Quốc 2019, 2019.
10. Stanley RP. Enumerative Combinatorics. Volume 62 of Cambridge Studies in Advanced Mathematics, 1999.
11. Min-25. Find Linear Recurrence with Polynomial Coefficients. <https://min-25.hatenablog.com/entry/2018/05/10/21280>.
12. Jin Ce. Phép toán và ứng dụng của hàm sinh. Trao đổi học viên WC2015, 2015.

Bàn về bài toán bước đi ngẫu nhiên trên mô hình đồ thị

Wang Xiuhuan, Trường Trung học số 7 Thành Đô

Tóm tắt

Bài toán bước đi ngẫu nhiên là một lớp bài toán thường gặp trong thi tin học, và cũng có ứng dụng rộng rãi trong đời sống thực tế, chẳng hạn thuật toán xếp hạng trang web của Google. Bài viết này xoay quanh bài toán ấy, lần lượt khảo sát từ ba góc độ: đồ thị lưới, đồ thị thưa và đồ thị tổng quát. Bài viết giới thiệu vài phương pháp thường dùng để giải lớp bài toán này, đồng thời so sánh ưu nhược điểm của chúng khi xử lý các kiểu bài khác nhau, với hy vọng giúp người đọc có đường hướng khi gặp bài toán dạng này.

1. Dẫn nhập

Trong các cuộc thi thuật toán những năm gần đây, số bài về bước đi ngẫu nhiên dần tăng lên. Lớp bài toán này biến hóa rất nhiều, cách giải tùy từng đề, nên thường khiến người giải khó bắt đầu.

Bài viết chủ yếu khảo sát từ ba góc độ: đồ thị lưới, đồ thị thưa và đồ thị tổng quát. Ta sẽ giới thiệu nhiều phương pháp giải các bài toán này và so sánh ưu nhược điểm của chúng trên các kiểu bài khác nhau, với hy vọng giúp người đọc có đường hướng khi gặp bài toán dạng này.

Mục 2 giới thiệu định nghĩa của bài toán bước đi ngẫu nhiên. Mục 3 giới thiệu hai phương pháp cho bài toán bước đi ngẫu nhiên trên đồ thị lưới, có độ phức tạp lần lượt là $O(n\sqrt{n})$ và $O(n^2)$. Mục 4 giới thiệu một phương pháp giải bài toán bước đi ngẫu nhiên trên đồ thị thưa với một cặp đỉnh xuất phát và kết thúc cố định, độ phức tạp $O(n^2)$. Mục 5 giới thiệu phương pháp giải bài toán trên đồ thị tổng quát cho mọi cặp đỉnh xuất phát và kết thúc, độ phức tạp $O(n^3)$. Mục 6 so sánh ưu nhược điểm của các phương pháp này, và mục 7 tổng kết toàn văn.

2. Định nghĩa

Cho một đồ thị đơn có hướng $G = (V, E)$, trong đó $V = \{v_1, v_2, \dots, v_{|V|}\}$, một đỉnh xuất phát $v_s \in V$ và một đỉnh kết thúc $v_t \in V$. Mỗi cạnh $e = (v_x, v_y)$ có trọng số dương w_e , thỏa

$$\forall v_x \in V \setminus \{v_t\}, \quad \sum_{(v_x, v_y) \in E} w_{(v_x, v_y)} = 1,$$

và từ mọi đỉnh v_x đều tồn tại một đường đi tới v_t . Một quân cờ bắt đầu từ đỉnh xuất phát; mỗi giây, nếu hiện đang ở v_x , nó chọn cạnh ra (v_x, v_y) với xác suất $w_{(v_x, v_y)}$ rồi đi tới v_y ; khi tới đỉnh kết thúc thì dừng. Cần tính kỳ vọng thời gian cần dùng¹².

Thật ra bài toán này cũng có thể viết dưới dạng ma trận. Định nghĩa ma trận P bởi

$$P_{x,y} = \begin{cases} w_{(v_x, v_y)}, & (v_x, v_y) \in E \text{ và } x \neq t, \\ 0, & (v_x, v_y) \notin E \text{ hoặc } x = t. \end{cases}$$

Đáp án cần tìm là

$$\sum_{k \geq 0} k \times (P^k)_{s,t},$$

¹Nếu trọng số bằng 0, có thể coi cạnh ấy không tồn tại.

²Để tiện mô tả, định nghĩa trong bài viết này hơi khác định nghĩa của bài toán bước đi ngẫu nhiên thực tế.

trong đó $(P^k)_{s,t}$ biểu thị xác suất lần đầu tới đỉnh kết thúc sau khi đi k bước. Khi đồ thị hữu hạn và mọi đỉnh đều tới được đỉnh kết thúc, từ định nghĩa của P có thể chứng minh mọi trị riêng của nó đều nhỏ hơn 1, nên đáp án chắc chắn hội tụ.

Để tiện mô tả, nếu không nói khác, trong bài này n luôn chỉ $|V|$, còn m chỉ $|E|$.

Ngoài ra, trong bài này, đồ thị thưa là đồ thị có số cạnh cùng bậc với số đỉnh.

3. Đồ thị lưới

Ví dụ 1. Circles of Waiting. Nguồn: Tinkoff Internship Warmup Round 2018 and Codeforces Round #475 (Div. 1), Problem E.

Một quân cờ ban đầu được đặt tại điểm $(0, 0)$ trên mặt phẳng tọa độ vuông góc. Mỗi giây quân cờ di chuyển ngẫu nhiên. Giả sử nó hiện ở (x, y) ; giây tiếp theo, nó có xác suất p_1 đi tới $(x - 1, y)$, xác suất p_2 đi tới $(x, y - 1)$, xác suất p_3 đi tới $(x + 1, y)$, và xác suất p_4 đi tới $(x, y + 1)$. Bảo đảm $p_1 + p_2 + p_3 + p_4 = 1$. Cần tính kỳ vọng thời gian cho tới khi nó đi tới một vị trí có khoảng cách Euclid tới gốc tọa độ lớn hơn R .

$$0 \leq R \leq 50, \quad p_1, p_2, p_3, p_4 > 0,$$

đáp án lấy modulo $10^9 + 7$.

3.1. Cách làm đơn giản

Gọi $f(i, j)$ là kỳ vọng thời gian để quân cờ, khi đang ở (i, j) , đi tới một vị trí có khoảng cách Euclid tới gốc tọa độ lớn hơn R . Phương trình chuyển là

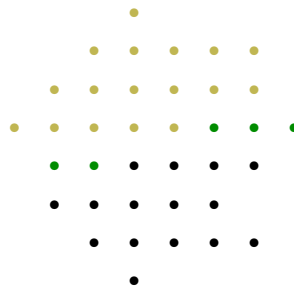
$$f(i, j) = \begin{cases} p_1 f(i - 1, j) + p_2 f(i, j - 1) + p_3 f(i + 1, j) + p_4 f(i, j + 1) + 1, & i^2 + j^2 \leq R, \\ 0, & i^2 + j^2 > R. \end{cases}$$

Do phép chuyển không có thứ tự tô-pô, cần dùng khử Gauss để giải. Độ phức tạp thời gian là $O(R^6)$, không thể qua bài này.

3.2. Khử trực tiếp

Chú ý rằng đa số hệ số trong các phương trình cần khử đều bằng 0. Khi khử chỉ tính trên những vị trí có giá trị khác 0 thì có thể giảm độ phức tạp [1].

Xét quá trình khử. Ta khử các phương trình theo thứ tự từ trên xuống dưới trong hệ tọa độ, và trong cùng một lớp thì từ trái sang phải. Những phương trình đã khử được tô vàng, các điểm kề với điểm vàng được tô xanh, các điểm còn lại được tô đen, như sơ đồ sau:



Tiếp theo ta cần khử phương trình tương ứng với ô xanh kế tiếp. Trong phương trình này, chỉ các hệ số của biến tương ứng với ô xanh và ô đen đầu tiên bên dưới nó là có thể khác 0; đồng thời, chỉ trong các

phương trình tương ứng với ô xanh và ô đen đầu tiên bên dưới nó, hệ số của biến tương ứng với ô hiện tại mới có thể khác 0.

Chú ý rằng chỉ có $O(R)$ ô xanh, nên thời gian khử một phương trình là $O(R^2)$. Tổng cộng chỉ có $O(R^2)$ phương trình, vì vậy độ phức tạp thời gian giảm còn $O(R^4)$, đủ qua bài này.

3.3. Phương pháp ẩn chính

Số phương trình và số biến đều là $O(R^2)$. Nếu có thể thu nhỏ quy mô xuống $O(R)$, thì khử Gauss đơn giản là đủ.

Đặt biến tương ứng với ô đầu tiên từ trái sang phải trên mỗi hàng làm ẩn chính; tổng cộng có $2R + 1$ ẩn chính. Ta tìm cách biểu diễn các biến tương ứng với những ô còn lại thành hàm tuyến tính theo các ẩn chính này. Xét từng cột từ trái sang phải. Với mỗi ô (i, j) trong cột hiện tại, chú ý rằng $f(i, j), f(i - 1, j), f(i, j - 1), f(i, j + 1)$ đều đã là hàm tuyến tính theo các ẩn chính. Chuyển về phương trình truy hồi, ta được

$$f(i + 1, j) = \frac{f(i, j) - p_1 f(i - 1, j) - p_2 f(i, j - 1) - p_4 f(i, j + 1) - 1}{p_3}.$$

Như vậy ta có biểu diễn tuyến tính của $f(i + 1, j)$ theo các ẩn chính. Nếu $(i + 1, j)$ đã có khoảng cách Euclid tới gốc tọa độ lớn hơn R , thì ta thu được một phương trình $f(i + 1, j) = 0$. Cuối cùng, ta sẽ có $2R + 1$ phương trình, rồi chỉ cần khử Gauss trên các phương trình ấy.

Trong giai đoạn truy đẩy các hàm tuyến tính theo ẩn chính, có $O(R^2)$ biến; truy đẩy một biến tốn $O(R)$ thời gian. Sau đó, quy mô bài toán đã được rút xuống $O(R)$. Hai phần đều có độ phức tạp $O(R^3)$, nên tổng độ phức tạp thời gian cũng là $O(R^3)$, đủ qua bài này.

3.4. So sánh hai cách làm

Ta so sánh hai cách làm ở nhiều khía cạnh.

Về độ phức tạp thời gian, phương pháp ẩn chính trên đồ thị lưới có độ phức tạp xấu nhất $O(n\sqrt{n})$ (đạt lớn nhất khi chiều dài và chiều rộng của lưới đều cùng bậc $O(\sqrt{n})$); phương pháp khử trực tiếp trên đồ thị lưới có độ phức tạp xấu nhất $O(n^2)$, nên phương pháp ẩn chính tốt hơn.

Về độ chính xác, với một số bài cần tính trên số thực thay vì lấy modulo, khử trực tiếp chính xác hơn phương pháp ẩn chính.

Về phạm vi áp dụng, hai cách làm phù hợp với những tình huống khác nhau.

Khi trong đồ thị lưới có chướng ngại, hoặc xác suất đi qua một số cạnh bằng 0, phương pháp ẩn chính cần thêm một ẩn chính cho mỗi chướng ngại hoặc mỗi cạnh xác suất 0. Khi số lượng chướng ngại hoặc cạnh xác suất 0 nhiều hơn $O(R)$, độ phức tạp của phương pháp ẩn chính sẽ tăng, trong khi độ phức tạp của khử trực tiếp vẫn không đổi.

Nhưng phương pháp ẩn chính còn có thể khử những phương trình chuyển tương tự trên đồ thị lưới, chẳng hạn

$$f(i, j) = p_1 f(i + 1, j) + p_2 f(i, j + 1) + p_3 f(\text{pre}(i, j)) + 1,$$

trong đó $\text{pre}(i, j) = (x, y)$, $x \leq i$, $y \leq j$, là giá trị do đề bài cho. Phân tích độ phức tạp của khử trực tiếp không áp dụng được cho mô hình này.

Ngoài ra, việc tính định thức của ma trận kề của đồ thị lưới cũng không thể dùng phương pháp ẩn chính, mà chỉ có thể dùng khử trực tiếp để tối ưu độ phức tạp thời gian.

Tóm lại, hai cách làm đều có sở trường riêng; cần phân tích bài cụ thể để chọn cách thích hợp.

4. Đồ thị thưa

Ví dụ 2. Expected Value. Nguồn: Ildar Gainullin Contest 1, Problem E, có chỉnh sửa.

Cho một đồ thị đơn vô hướng liên thông phẳng $G = (V, E)$. Một quân cờ ban đầu được đặt tại v_1 ; mỗi giây quân cờ chọn đều một cạnh nối với đỉnh hiện tại rồi đi tới đỉnh ở đầu kia. Cần tính kỳ vọng thời gian để tới v_n .

$$n \leq 2000,$$

đáp án lấy modulo p , trong đó p là một số nguyên tố được sinh ngẫu nhiên trong khoảng $[10^9, 1.01 \times 10^9]$.

Một kết luận là số cạnh của đồ thị phẳng cùng bậc với số đỉnh. Cụ thể, ta có:

Định lý 4.1. Một đồ thị phẳng có n đỉnh ($n \geq 3$) thì số cạnh m không vượt quá $3n - 6$ [2].

Vì cách làm của bài này có thể mở rộng sang mọi đồ thị thưa, nên phần thảo luận sau chỉ dựa trên điều kiện số cạnh và số đỉnh cùng bậc, không cần các tính chất khác của đồ thị phẳng.

4.1. Kiến thức cơ sở

Định nghĩa 4.1. Mọi đa thức $p(\lambda)$ thỏa $p(A) = 0$ được gọi là đa thức triệt tiêu của ma trận A .

Định nghĩa 4.2. Ký hiệu I_n là ma trận đơn vị cấp n . Định nghĩa đa thức đặc trưng của một ma trận $n \times n$ là

$$p(\lambda) = \det(\lambda I_n - A),$$

trong đó $\det$ là định thức của ma trận.

Dễ thấy đa thức đặc trưng của một ma trận cấp n có bậc không vượt quá n .

Định lý 4.2 (Cayley–Hamilton [3]). Đa thức đặc trưng của mọi ma trận đều là đa thức triệt tiêu của nó.

Vì vậy, đa thức triệt tiêu có bậc nhỏ nhất của một ma trận cấp n cũng có bậc không vượt quá n .

4.2. Giải bài toán ban đầu

Chú ý rằng kỳ vọng thời gian đi được là

$$E(t) = \sum_{i \geq 0} \Pr[t > i].$$

Nếu ta tính được xác suất sau khi đi i bước mà vẫn chưa kết thúc, thì cộng trên mọi $i \geq 0$ sẽ được đáp án.

Gọi $f(i, j)$ là xác suất sau khi đi i bước, quân cờ đang dừng ở v_j và chưa từng đi tới v_n . Khi đó

$$f(i, j) = \sum_{(v_k, v_j) \in E} \frac{f(i-1, k)}{\deg_k} \quad (j \neq n),$$

trong đó $\deg_k$ là bậc của v_k .

Chú ý rằng phép chuyển của f không phụ thuộc vào i , nên có thể xem mỗi lần chuyển là nhân thêm một ma trận, tức $f_{i+1} = f_i M$. Vì đa thức triệt tiêu nhỏ nhất của M có bậc không vượt quá n , nên công thức truy hồi ngắn nhất của f cũng có độ dài không vượt quá n . Do đó

$$\Pr[t > i] = \sum_{j=1}^{n-1} f(i, j)$$

cũng có công thức truy hồi ngắn nhất dài không vượt quá n . Ta có thể tính $\Pr[t > 0], \Pr[t > 1], \dots, \Pr[t > 3n]$ trong $O(nm)$ thời gian, rồi dùng thuật toán Berlekamp–Massey [4] để tìm công thức truy hồi ngắn nhất của $\Pr[t > i]$ trong $O(n^2)$ thời gian.

Xét cách tính hàm sinh của một dãy truy hồi tuyến tính bậc k . Không mất tính tổng quát, giả sử với $i \geq i_0$ ta có

$$a_i = \sum_{j=1}^k c_j a_{i-j}.$$

Gọi hàm sinh của a và c lần lượt là $A(x)$ và $C(x)$. Khi đó

$$A(x) = A(x)C(x) + A_0(x),$$

trong đó $A_0(x)$ do các hạng $i < i_0$ quyết định.

Quay lại bài toán ban đầu. Vì ta tìm được công thức truy hồi ngắn nhất của $\Pr[t > i]$, nên có thể tìm $C(x)$ và $A_0(x)$ (định nghĩa như đoạn trước). Chuyển về được

$$A(x) = \frac{A_0(x)}{1 - C(x)}.$$

Giá trị cần tìm là $\sum_{i \geq 0} [x^i]A(x)$; để thấy giá trị này chính là $A(1)$. Thay $x = 1$ vào rồi giải là xong. Vì modulo là số nguyên tố ngẫu nhiên, có thể coi mẫu số sẽ không bằng 0.

Như vậy, ta giải được bài này trong $O(nm + n^2)$ thời gian. Do số đỉnh và số cạnh cùng bậc, trong bài này độ phức tạp có thể coi là $O(n^2)$.

Ngoài ra, bài này còn có một cách làm khác; xem tài liệu tham khảo [5].

5. Đồ thị tổng quát

Ví dụ 3. Frank. Nguồn: 2018 ACM-ICPC Asia Nanjing Regional Contest, Problem F, có chỉnh sửa.

Cho một đồ thị đơn có hướng liên thông mạnh $G = (V, E)$. Với mọi $1 \leq s \leq n, 1 \leq t \leq n, s \neq t$, hãy trả lời bài toán sau.

Một quân cờ ban đầu được đặt tại v_s ; mỗi giây quân cờ chọn đều một cạnh ra của đỉnh hiện tại rồi đi tới đỉnh mà cạnh ấy trở tới. Cần tính kỳ vọng thời gian để tới v_t .

$$3 \leq n \leq 400.$$

5.1. Phân tích và chuyển hóa

Gọi $p_{i,j}$ là xác suất quân cờ, khi ở v_i , chọn cạnh ra (v_i, v_j) để đi tới v_j ; đặc biệt, nếu cạnh ra không tồn tại thì xác suất bằng 0. Gọi $f_{i,j}$ là kỳ vọng thời gian để đi ngẫu nhiên từ v_i tới v_j ; đặc biệt, $f_{i,i} = 0$. Khi $i \neq j$, phương trình chuyển là

$$f_{i,j} = 1 + \sum_{1 \leq k \leq n} p_{i,k} f_{k,j}.$$

Khi $i = j$, gọi g_i là kỳ vọng thời gian để bắt đầu từ v_i , đi ngẫu nhiên và lần đầu quay lại v_i . Khi đó

$$f_{i,i} = 1 - g_i + \sum_{1 \leq k \leq n} p_{i,k} f_{k,i}.$$

Để dễ quan sát, ta viết phương trình chuyển dưới dạng ma trận. Gọi P là ma trận chuyển của đồ thị, F là ma trận đáp án, I là ma trận đơn vị cấp n , J là ma trận toàn 1 cấp n , và G là ma trận cấp n thỏa $G_{i,i} = g_i$, các vị trí khác bằng 0. Khi đó

$$F = J - G + PF.$$

Nếu tính được G , ta chỉ cần giải phương trình [6]

$$(I - P)F = J - G.$$

5.2. Cách tính G

Định nghĩa 5.1. Định nghĩa phân bố dừng [7] của một ma trận chuyển cấp n là một vectơ n chiều π , thỏa

$$\sum_{i=1}^n \pi_i = 1, \quad \pi P = \pi.$$

Giá trị ở mỗi chiều của π đều nằm trong đoạn $[0, 1]$.

Ta rất dễ tìm được ý nghĩa thực tế của phân bố dừng. Nếu tại một thời điểm nào đó quân cờ dừng ở v_i với xác suất π_i , thì ở mọi thời điểm về sau quân cờ vẫn thỏa phân bố xác suất này. Ta có thể dùng khử Gauss để giải hệ và tìm π trong $O(n^3)$ thời gian. Vậy π có quan hệ gì với G ?

Định lý 5.1. Với mọi $1 \leq i \leq n$, ta có

$$\pi_i g_i = 1.$$

Chứng minh. Từ $F = J - G + PF$, chuyển vế được

$$G = PF + J - F.$$

Nhân đồng thời hai vế với π ở bên trái:

$$\pi G = \pi PF + \pi J - \pi F.$$

Theo định nghĩa của π , ta có $\pi P = \pi$, do đó

$$\pi G = \pi J.$$

Vì vậy

$$\pi_i g_i = \sum_{j=1}^n \pi_j = 1.$$

Mệnh đề được chứng minh. $\square$

Vậy bằng cách đưa vào phân bố dừng, ta có thể tính G trong $O(n^3)$ thời gian.

5.3. Giải bài toán ban đầu

Trong quá trình giải phương trình, ta gặp một vấn đề: $I - P$ không đầy hạng, nên không thể giải bằng cách nhân ma trận nghịch đảo.

Định nghĩa 5.2. Định nghĩa một cây khung có hướng gốc $v_r \in V$ của đồ thị có hướng $G = (V, E)$ là một đồ thị con $T = (V, A)$ của G thỏa:

1. Với mọi $v_i \neq v_r$, bậc ra của v_i bằng 1.
2. Bậc ra của v_r bằng 0.
3. Trong T không tồn tại chu trình.

Bổ đề 5.1 (Định lý cây ma trận trên đồ thị có hướng [8]). Với một đồ thị có hướng G , gọi D là ma trận bậc ra của nó, tức

$$D_{i,i} = d_i, \quad D_{i,j} = 0 \quad (i \neq j),$$

trong đó d_i là bậc ra của v_i ; gọi A là ma trận kề của nó. Khi đó số cây khung có hướng gốc v_r bằng định thức của $D - A$ sau khi bỏ hàng r và cột r .

Định lý 5.2. Với ma trận chuyển P của một đồ thị liên thông mạnh $G = (V, E)$, hạng của $I - P$ bằng $n - 1$.

Chứng minh. Vì nhân một hàng của ma trận với một hằng số khác 0 không làm đổi hạng, ta nhân hàng thứ i của $I - P$ với bậc ra của v_i , thu được một ma trận mới L . Chỉ cần chứng minh hạng của L bằng $n - 1$.

Do tổng trên mỗi hàng của L đều bằng 0, cộng tất cả các vectơ cột của L sẽ thu được vectơ không; tức các vectơ này phụ thuộc tuyến tính. Vì thế hạng của L không phải n .

Để thấy L bằng ma trận bậc ra của G trừ đi ma trận kề của nó. Theo bổ đề 5.1, định thức của L sau khi bỏ hàng i và cột i biểu thị số cây khung có hướng gốc v_i .

Vì G liên thông mạnh, với mọi đỉnh được chọn làm gốc, số cây khung có hướng đều khác 0. Tức là sau khi bỏ hàng i và cột i , L vẫn đầy hạng.

Vì thêm một cột không làm hạng giảm, mọi vectơ hàng của L sau khi bỏ hàng i là độc lập tuyến tính. Do đó hạng của L là $n - 1$. $\square$

Quay lại bài toán ban đầu, xét cách giải phương trình trong bài. Để tiện, ta viết phương trình dưới dạng $AX = B$, trong đó A, B đã biết, cần tìm X . Do A không đầy hạng, nghiệm có vô số; trước hết ta tìm một nghiệm riêng.

Khử Gauss đồng thời trên A và B . Khử $n - 1$ hàng đầu của A về dạng chỉ còn đường chéo chính và cột thứ n có giá trị, còn hàng cuối cùng toàn 0, tức dạng sau:

$$\begin{bmatrix} 1 & 0 & 0 & \cdots & 0 & a_1 \\ 0 & 1 & 0 & \cdots & 0 & a_2 \\ 0 & 0 & 1 & \cdots & 0 & a_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 1 & a_{n-1} \\ 0 & 0 & 0 & \cdots & 0 & 0 \end{bmatrix} X = \begin{bmatrix} b_{1,1} & b_{1,2} & b_{1,3} & \cdots & b_{1,n-1} & b_{1,n} \\ b_{2,1} & b_{2,2} & b_{2,3} & \cdots & b_{2,n-1} & b_{2,n} \\ b_{3,1} & b_{3,2} & b_{3,3} & \cdots & b_{3,n-1} & b_{3,n} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ b_{n-1,1} & b_{n-1,2} & b_{n-1,3} & \cdots & b_{n-1,n-1} & b_{n-1,n} \\ 0 & 0 & 0 & \cdots & 0 & 0 \end{bmatrix}.$$

Đặt $X_{n,i} = 0$, ta giải được một nghiệm riêng, ký hiệu là Y . Tiếp theo ta cần điều chỉnh nghiệm riêng thành nghiệm thật sự.

Chú ý rằng $X_{i,i} = 0$. Xét theo ý nghĩa tổ hợp, ta có

$$Y_{i,j} = 1 + Y_{j,j} + \sum_{k=1}^n P_{i,k} X_{k,j},$$

và dễ suy ra

$$X_{i,j} = Y_{i,j} - Y_{j,j}.$$

Cuối cùng, ta giải được bài toán này trong $O(n^3)$ thời gian.

6. So sánh và thảo luận

Về độ phức tạp thời gian, trên đồ thị lưới có phương pháp ẩn chính $O(n\sqrt{n})$; thuật toán trên đồ thị thưa là $O(n^2)$; còn trên đồ thị tổng quát, độ phức tạp cao hơn, là $O(n^3)$.

Về góc độ bài toán được giải, thuật toán trên đồ thị lưới có thể giải cho trường hợp mọi đỉnh trong đồ thị đều làm đỉnh xuất phát; thuật toán trên đồ thị thưa chỉ có thể giải nhanh cho một đỉnh xuất phát và một đỉnh kết thúc; thuật toán trên đồ thị tổng quát thì giải cho mọi cặp đỉnh làm xuất phát và kết thúc. Nếu các thuật toán trên đồ thị lưới, đồ thị thưa phải giải cho nhiều đỉnh kết thúc, hoặc thuật toán trên đồ thị thưa phải giải cho nhiều đỉnh xuất phát, thì cần thêm thời gian.

Về phạm vi áp dụng, vì đồ thị lưới là tập con của đồ thị thưa, còn đồ thị thưa là tập con của đồ thị tổng quát, nên thuật toán trên đồ thị lưới có phạm vi áp dụng nhỏ nhất, còn thuật toán trên đồ thị tổng quát có phạm vi áp dụng lớn nhất.

Về độ chính xác, do thuật toán Berlekamp–Massey có độ chính xác rất kém, nên trong các bài cần tính trên số thực, thông thường không thể dùng thuật toán trên đồ thị thưa; còn thuật toán trên đồ thị lưới và đồ thị tổng quát đều có thể chọn dùng.

Tóm lại, vài thuật toán này đều có ưu nhược điểm riêng. Việc dùng thuật toán nào thường tùy bài; khi giải lớp bài này, thí sinh nên phân tích tính chất của đề rồi chọn phương pháp phù hợp nhất.

7. Tổng kết

Bài viết đã nghiên cứu bài toán bước đi ngẫu nhiên trên mô hình đồ thị, lần lượt giới thiệu vài thuật toán khác nhau trên đồ thị lưới, đồ thị thưa và đồ thị tổng quát. Chú ý rằng phần lớn các thuật toán này được thảo luận trên nền tảng ma trận; điều đó cho thấy ma trận là một công cụ mạnh để giải bài toán bước đi ngẫu nhiên. Với bài toán bước đi ngẫu nhiên, sau khi chuyển thành dạng ma trận, ta thường sẽ dễ bắt tay giải hơn.

Đồng thời, các thuật toán được nhắc tới trong bài không chỉ dùng được cho bước đi ngẫu nhiên, mà còn có thể gợi mở cho những bài toán khác: khử trực tiếp trong đồ thị lưới gợi ý rằng với một số ma trận thưa có tính chất đặc biệt, ta có thể dùng một số cắt tía để giảm độ phức tạp; phương pháp ẩn chính có thể dùng cho một lớp bài toán giải hệ phương trình; thuật toán trong phần đồ thị thưa gợi ý cách dùng truy hồi tuyến tính để giải một số bài toán liên quan tới ma trận; phân bố dừng trong phần đồ thị tổng quát là công cụ mạnh để xử lý các bài toán liên quan tới xích Markov, và nó còn nhiều tính chất đáng khai thác.

Thật ra bài toán bước đi ngẫu nhiên còn rất nhiều điểm đáng nghiên cứu. Mong rằng bài viết có thể đóng vai trò gợi mở và thu hút thêm nhiều độc giả nghiên cứu lớp bài toán này.

8. Lời cảm ơn

Xin cảm ơn Hội Máy tính Trung Quốc đã cung cấp nền tảng học tập và trao đổi.

Xin cảm ơn thầy Lin Yang và thầy Zhang Junliang của Trường Trung học số 7 Thành Đô đã quan tâm và hướng dẫn tôi.

Xin cảm ơn huấn luyện viên Zhang Ruizhe của đội tuyển tập huấn quốc gia đã hướng dẫn.

Xin cảm ơn đàn anh Yang Jingqin đã giúp đỡ tôi.

Xin cảm ơn các tiền bối Chen Songyang, Tang Jingzhe và Du Yuhao đã truyền cảm hứng và giúp đỡ tôi trong quá trình viết bài.

Xin cảm ơn tiền bối Chen Songyang, tiền bối Tang Jingzhe và bạn Fang Lixing đã kiểm tra bản thảo.

Tài liệu tham khảo

1. Romanov, V. Editorial Tinkoff Internship Warmup Round 2018 and Codeforces Round #475 (Div. 1 + Div. 2). Codeforces, <https://codeforces.com/blog/entry/58991>.
2. Wikipedia contributors. Planar graph. Wikipedia, https://en.wikipedia.org/wiki/Planar_graph.
3. Hamilton, W. R. Lectures on Quaternions. Dublin, 1853.
4. Massey, J. L. Shift-register synthesis and BCH decoding. IEEE Transactions on Information Theory, IT-15(1): 122–127, 1969. doi:10.1109/TIT.1969.1054260.
5. Zhong Ziqian. Tính chất và ứng dụng của hai loại dãy truy hồi. Tập luận văn ứng viên đội tuyển quốc gia IOI Trung Quốc 2019, 2019.
6. Tang, J. 2018-2019 ACM-ICPC, Asia Nanjing Regional Contest (Online Mirror on Gym). Codeforces, <https://codeforces.com/blog/entry/63057>.
7. Wikipedia contributors. Stationary Distribution. Wikipedia, https://en.wikipedia.org/wiki/Stationary_Distribution.
8. Chaiken, S.; Kleitman, D. Matrix Tree Theorems. Journal of Combinatorial Theory, Series A, 24(3): 377–381, 1978. doi:10.1016/0097-3165(78)90067-5, ISSN 0097-3165.

Báo cáo ra đề *Bài toán nhỏ dễ*

Yang Junzhao, Trường Ngoại ngữ Nam Kinh

Tóm tắt

Bài viết này giới thiệu bài thứ ba trong vòng tự kiểm tra thứ năm của đội ứng viên, do tác giả ra đề. Đây là một bài cấu trúc dữ liệu lấy bối cảnh duy trì sự thay đổi mực nước trong các bể nước. Bài toán cần trước hết chuyển mô hình vật lý của sự biến đổi mực nước thành một mô hình toán học để duy trì, rồi dùng kỹ thuật cấu trúc dữ liệu để tối ưu và giải quyết.

1. Ý chính đề bài

1.1. Mô tả đề bài

Có n bể nước có cùng diện tích đáy, xếp thành một hàng. Hai bể kề nhau dùng chung một vách ngăn; chiều cao vách ngăn giữa bể thứ i và bể thứ $i + 1$ là b_i . Vách ở ngoài cùng bên trái và bên phải có thể xem là cao vô hạn. Ban đầu tất cả các bể đứng yên, mực nước ban đầu của bể thứ i là a_i . Bạn cần duy trì hai loại thao tác sau:

- Cho x và h , khoan một lỗ nhỏ ở độ cao h trên vách ngăn giữa bể thứ x và bể thứ $x + 1$. Sau đó mực nước của một số bể sẽ thay đổi; bạn phải chờ cho đến khi các bể ấy đứng yên trở lại.
- Cho x , hỏi mực nước hiện tại của bể thứ x .

1.2. Dạng nhập

Dữ liệu được đọc từ chuẩn nhập.

Dòng đầu gồm hai số nguyên n, q , lần lượt là số bể và số thao tác.

Dòng thứ hai gồm n số thực a_i , cách nhau bởi dấu cách, biểu thị mực nước ban đầu.

Dòng thứ ba gồm $n - 1$ số thực b_i , cách nhau bởi dấu cách, biểu thị chiều cao vách ngăn ban đầu.

Tiếp theo là q dòng, mỗi dòng mô tả một thao tác.

1 x h biểu thị thao tác loại một, bảo đảm $1 \leq x < n$. Chú ý h là số thực.

2 x biểu thị thao tác loại hai, bảo đảm $1 \leq x \leq n$.

Mọi số thực xuất hiện trong đầu vào có không quá bảy chữ số sau dấu thập phân.

1.3. Dạng xuất

In ra chuẩn xuất.

Với mỗi thao tác loại hai, in một dòng là đáp án.

Dòng cuối cùng in n số thực cách nhau bởi dấu cách, biểu thị mực nước của từng bể sau khi mọi thao tác kết thúc.

Sai số tuyệt đối trong phạm vi 10^{-7} được chấp nhận.

1.4. Gợi ý và thuyết minh bổ sung

Trong bài này, ta dùng một mô hình lý tưởng. Tức là ta giả sử thể tích nước không đổi, đồng thời bỏ qua sức căng bề mặt, ma sát, v.v. Có thể hiểu rằng sự thay đổi mực nước sau khi khoan lỗ phù hợp với trực giác đời sống. Cụ thể, với hai mực nước kề nhau a_i, a_{i+1} , nếu vách ngăn giữa chúng có một lỗ ở độ cao h (hoặc chiều cao vách ngăn ban đầu là h), thì nếu $a_i > h$ và $a_i > a_{i+1}$, sẽ có nước chảy từ i sang $i + 1$. Đối xứng, nếu $a_{i+1} > h$ và $a_{i+1} > a_i$, sẽ có nước chảy từ $i + 1$ sang i . Nước chảy từ x sang y nghĩa là a_x liên tục giảm, a_y liên tục tăng, và tổng của chúng được giữ nguyên.

1.5. Giới hạn và quy ước

Với mọi dữ liệu, $1 \leq n, q \leq 100000$, $0 \leq a_i, h \leq 100$, và với $1 \leq i < n$ có $b_i \geq \max(a_i, a_{i+1})$. Mọi số thực xuất hiện trong đầu vào có không quá bảy chữ số sau dấu thập phân.

- Subtask 1 (1 điểm): $1 \leq n, q \leq 10$;
- Subtask 2 (10 điểm): $1 \leq n, q \leq 300$;
- Subtask 3 (10 điểm): $1 \leq n, q \leq 2000$, với $i > 1$ có $a_i = 0$;
- Subtask 4 (10 điểm): $1 \leq n, q \leq 2000$;
- Subtask 5 (30 điểm): với $i > 1$ có $a_i = 0$;
- Subtask 6 (39 điểm): không có ràng buộc đặc biệt.

Giới hạn thời gian: 4s.

Giới hạn bộ nhớ: 1024MB.

2. Phân tích lời giải

2.1. Thuật toán một

Đây là một bài cấu trúc dữ liệu. Trước hết xét mô phỏng trực tiếp theo đề. Bài toán yêu cầu duy trì quá trình nước chảy từ nơi cao xuống nơi thấp. Nhưng khác với một bài cấu trúc dữ liệu thông thường, vấn đề không đơn giản như vậy, vì mô tả đề không phải là ngôn ngữ đã được chương trình hóa; do đó trước hết ta cần xây dựng mô hình.

Vì vách ngăn chỉ liên quan tới giá trị nhỏ hơn giữa chiều cao ban đầu của nó và độ cao lỗ khoan, nên trong phần sau ta trực tiếp xem chiều cao vách ngăn (tức b_i) là giá trị nhỏ hơn ấy.

Gợi ý trong đề đã đưa ra một mô hình khả dĩ. Mỗi lần chỉ xét hai bể kề nhau; nếu chúng ở trạng thái mất cân bằng thì điều chỉnh chúng về cân bằng, cho đến khi không còn hai bể kề nhau nào mất cân bằng.

Mô hình này không dễ duy trì. Xét trường hợp có ba bể, nước từ bể thứ nhất chảy sang hai bể sau; cuối cùng mực nước của cả ba bể đáng ra phải bằng nhau, nhưng vì mỗi lần chỉ điều chỉnh mực nước của hai bể kề nhau, không thể làm cho chúng bằng nhau trong hữu hạn bước, quá trình này có thể tiếp diễn vô hạn. Tuy vậy, vì ta không cần đáp án chính xác mà chỉ cần bảo đảm chín chữ số chính xác (bao gồm hàng đơn vị, hàng chục và bảy chữ số sau dấu thập phân), nên khi hai độ cao rất gần nhau ta có thể dừng lại. Từ đó có thuật toán một:

Thuật toán một. Đặt $\varepsilon = 10^{-8}$. Với $1 \leq i < n$, gọi vị trí này là mất cân bằng khi và chỉ khi

$$a_i \geq \max(b_i, a_{i+1}) + \varepsilon \quad \text{hoặc} \quad a_{i+1} \geq \max(b_i, a_i) + \varepsilon.$$

Với mỗi lần sửa đổi, ta liên tục tìm trạng thái mất cân bằng và điều chỉnh, cho đến khi không còn trạng thái mất cân bằng. Nếu $a_i \geq \max(b_i, a_{i+1}) + \varepsilon$, lượng nước thay đổi là

$$\min\left(a_i - b_i, \frac{a_i - a_{i+1}}{2}\right),$$

ngược lại lượng nước thay đổi là

$$\min\left(a_{i+1} - b_i, \frac{a_{i+1} - a_i}{2}\right).$$

Độ phức tạp thuật toán: $O(q \times \text{một hằng số lớn phụ thuộc vào độ chính xác})$.

Điểm kỳ vọng: có thể qua subtask một, kỳ vọng 1 điểm.

2.2. Thuật toán hai

Ta cần một thuật toán thời gian đa thức, có độ phức tạp không phụ thuộc vào độ chính xác. Có thể quan sát một số tính chất:

- Sau mỗi lần sửa chiều cao vách ngăn, xét mực nước của hai bể kề với vách ấy, nước chỉ chảy từ cao xuống thấp. Vì vậy hoặc không có gì xảy ra, hoặc nước chảy từ trái sang phải, hoặc nước chảy từ phải sang trái.
- Nước chảy từ trái sang phải và từ phải sang trái là đối xứng. Khi phân tích tính chất bên dưới, không mất tính tổng quát giả sử nước chảy từ trái sang phải; còn khi bàn cách duy trì thông tin thì xét ảnh hưởng của cả hai hướng.

Vấn đề chính của thuật toán một là không giải quyết được cân bằng của nhiều bể. Ta xét cải tiến thuật toán một. Gọi một đoạn $[l, r]$ là liên thông khi và chỉ khi đồng thời thỏa ba điều kiện sau:

- $1 \leq l \leq r \leq n$;
- với $l \leq i \leq r$, có $a_i = a_l$;
- với $l \leq i < r$, có $b_i < \max(a_i, a_{i+1})$.

Chú ý rằng khi mực nước thay đổi, mọi bể trong cùng một bình thông nhau thay đổi cùng lúc, tức cùng tăng hoặc cùng giảm một thể tích nước như nhau. Dựa trên đoạn liên thông, ta thiết kế thuật toán hai:

Thuật toán hai. Thay vì xét hai bể kề nhau như cách trước, ta xét hai đoạn liên thông kề nhau, rồi để nước chảy từ cao xuống thấp cho đến khi điều kiện liên thông bị phá vỡ hoặc giữa chúng không còn chênh lệch độ cao. Trong cài đặt cụ thể, cần hỗ trợ:

- gộp hai đoạn liên thông;
- tách một đoạn liên thông thành hai đoạn liên thông;
- cộng/trừ đồng loạt trên một đoạn liên thông;
- hỏi chiều cao vách ngăn lớn nhất trong một đoạn liên thông.

Độ phức tạp thuật toán: dễ thấy sau mỗi lần điều chỉnh, đầu trái hoặc đầu phải của đoạn liên thông đang ở phía cao hơn sẽ dịch sang phải ít nhất một vị trí, nên sau mỗi thao tác chỉ cần điều chỉnh $O(n)$ lần. Vì vậy độ phức tạp của mỗi thao tác sửa là $O(n \times \text{độ phức tạp điều chỉnh})$. Tùy cách cài đặt, mỗi lần điều chỉnh có thể đạt $O(n)$, $O(\log n)$ hoặc $O(1)$. Trường hợp $O(1)$ có thể dùng hàng đợi đơn điệu, và mỗi thao tác sửa cần thêm $O(n)$. Tổng độ phức tạp có thể đạt $O(qn^2)$, $O(qn \log n)$ hoặc $O(qn)$.

Điểm kỳ vọng: tùy cách cài đặt, có thể qua hai subtask đầu hoặc bốn subtask đầu, kỳ vọng 11 đến 31 điểm.

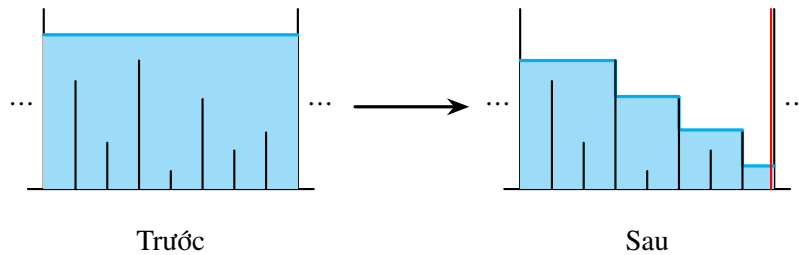
2.3. Thuật toán ba

Tổng số lần gộp hoặc tách đoạn liên thông trong thuật toán hai có thể đạt bậc hai, nên không thể qua hai subtask cuối. Chú ý rằng tuy trong thuật toán hai vẫn còn thao tác dư thừa (xét một dãy bể dạng bậc thang, nước ở bên trái có thể chảy xuôi theo bậc thang đến phía dưới bên phải, còn các đoạn liên thông ở giữa không thay đổi), ta vẫn có thể dựng phương án sao cho mỗi thao tác hoặc gộp $O(n)$ đoạn liên thông, hoặc tách một đoạn liên thông thành $O(n)$ đoạn liên thông. Nói cách khác, tồn tại một phương án khiến tổng các trị tuyệt đối của hiệu số lượng đoạn liên thông giữa hai thao tác kế nhau đạt bậc hai.

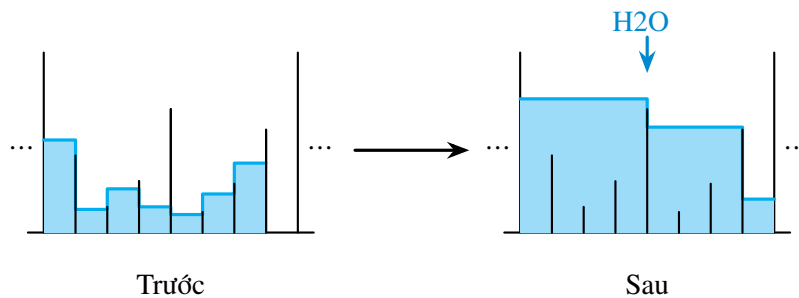
Ta suy nghĩ bài toán từ một góc nhìn khác. Có thể thấy nếu trong một thao tác mà duy trì biến đổi của các bể theo thời gian thì sẽ rất rắc rối; ta xét cách tính trực tiếp kết quả sau khi thay đổi.

Sau khi phân tích bình tĩnh, có ba tình huống cần xử lý:

1. Sau khi nước chảy đi, để lại một đoạn “bậc thang”.
2. Khi nước chảy qua, nó lấp đầy một số “hố”.
3. Nước chảy xuống bị vách ngăn chặn lại, tạo thành một “hồ”.



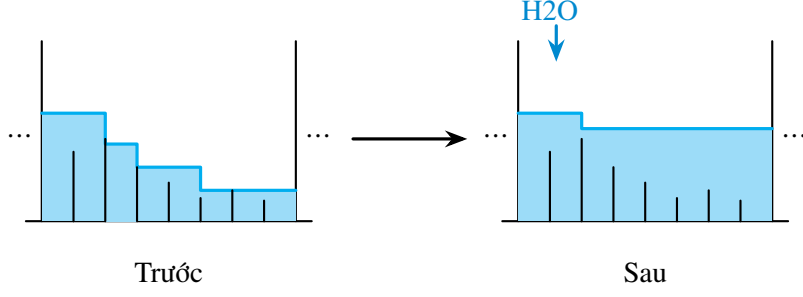
Hình 1: Sau khi nước chảy đi, để lại một đoạn bậc thang.



Hình 2: Khi nước chảy qua, nó lấp đầy một số hố.

Ta có thể chia toàn bộ quá trình biến đổi thành ba giai đoạn theo ba hình trên. Giả sử ta sửa chiều cao vách ngăn giữa bể thứ x và bể thứ $x + 1$ thành b'_x . Đặt $h = \max(b'_x, a_{x+1})$. Khi đó để nước chảy từ trái sang phải cần có $a_x > h$.

- Giai đoạn một: giả sử mực nước của bể thứ $x + 1$ là âm vô cùng, ta cần tính có bao nhiêu nước sẽ chảy đi. Chú ý chỉ mực nước trong đoạn liên thông mới thay đổi. Gọi đầu trái của đoạn liên thông là l . Gọi thể tích nước chảy đi là V . (Xem Hình 1.)



Hình 3: Nước bị vách ngăn chặn lại và tạo thành hồ.

- Giai đoạn hai: giả sử V đơn vị thể tích nước chảy từ bể thứ l sang phải, tìm r lớn nhất sao cho thể tích cần để lấp các hố từ l đến r không vượt quá V . (Xem Hình 2.)
- Giai đoạn ba: gọi lượng nước còn lại sau khi lấp hố là V' . Ta cần đổ lượng nước còn lại vào một số bể phía bên phải. Điều này tương đương với tìm h' sao cho trong đoạn $[l, r]$, thể tích cần để nâng mọi bể có mực nước nhỏ hơn h' lên đúng h' bằng V' . (Xem Hình 3.)

Chú ý rằng ba giai đoạn này không chỉ cần truy vấn mà còn cần sửa mực nước. Tuy các thao tác trong các giai đoạn này nhìn có vẻ khó tối ưu, suy nghĩ kỹ sẽ thấy chúng có điểm tương tự. Xét thao tác sau:

Với một đoạn $[l, r]$, đặt

$$FL(l, r, h) = \sum_{i=l}^r \max \left(\max_{j=i}^{r-1} b_j, h \right).$$

Đối xứng, đặt

$$FR(l, r, h) = \sum_{i=l}^r \max \left(\max_{j=l}^{i-1} b_j, h \right).$$

Ký hiệu $ML(l, r, h)$ là thao tác sửa sau: với $l \leq x \leq r$, đặt

$$a_x = \max \left(\max_{i=x}^{r-1} b_i, h \right).$$

Đối xứng, $MR(l, r, h)$ là thao tác sửa sau: với $l \leq x \leq r$, đặt

$$a_x = \max \left(\max_{i=l}^{x-1} b_i, h \right).$$

Nếu biểu diễn hậu tố max (hoặc tiền tố max) của chiều cao vách ngăn trên hình, ta gọi đường giống như đường đồng mức này là đường viền; tổng mực nước trên đường viền, tức tổng thể tích nước dưới đường viền, được gọi là mực nước đường viền của đoạn này. Ý nghĩa thực tế của hàm này (lấy FL làm ví dụ) là: giả sử ban đầu trong đoạn không có nước, vách ngăn ở hai bên đoạn cao vô hạn, ta chậm rãi đổ nước vào bể ngoài cùng bên trái cho đến khi mực nước ở bể ngoài cùng bên phải đạt h ; khi đó hàm cho biết thể tích nước cần dùng.

Lúc này ba giai đoạn có thể được mô tả hình thức bằng mực nước đường viền. Dùng $SUM(l, r)$ biểu thị tổng mực nước từ bể thứ l đến bể thứ r trong đoạn $[l, r]$.

- Giai đoạn một: tìm đầu trái l của đoạn liên thông. Tính $V = SUM(l, x) - FL(l, x, h)$. Thực hiện thao tác $ML(l, x, h)$.
- Giai đoạn hai: tìm r lớn nhất sao cho $FL(l, r, a_r) - SUM(l, r) \leq V$, đặt $V' = V - FL(l, r, a_r) + SUM(l, r)$. Thực hiện thao tác $ML(l, r, a_r)$.
- Giai đoạn ba: tìm h lớn nhất sao cho $FL(l, r, h) - SUM(l, r) \leq V'$. Thực hiện thao tác $ML(l, r, h)$.

Ở đây giai đoạn ba dùng tính đơn điệu của hàm FL theo h , tức nếu $h \leq h'$ thì $FL(l, r, h) \leq FL(l, r, h')$.

Ta phát hiện có thể hợp nhất đẹp ba quá trình này thành một quá trình:

- Tìm đầu trái l của đoạn liên thông. Tìm r lớn nhất, rồi tìm h lớn nhất, sao cho $FL(l, r, h) \leq \text{SUM}(l, r)$. Sau đó thực hiện $ML(l, r, h)$.

Chú ý ở đây phải bảo đảm r là lớn nhất trước, rồi mới bảo đảm h là lớn nhất; nếu không sẽ xuất hiện trạng thái mất cân bằng. Hàm FL ở đây còn có một tính đơn điệu nữa, tức đơn điệu theo r : nếu $r \leq r'$ thì

$$FL(l, r, a_r) \leq FL(l, r', a_{r'}).$$

Ta thấy nếu có một cấu trúc dữ liệu hỗ trợ truy vấn nhanh FL, FR và hỗ trợ nhanh các thao tác ML, MR , chỉ cần hai lần chặt nhị phân là giải được bài toán.

Thuật toán ba. Dùng phương pháp trên. Tính FL, FR bằng vét cạn $O(n)$, và sửa ML, MR bằng vét cạn.

Độ phức tạp thuật toán: vì mỗi thao tác sửa cần chặt nhị phân, độ phức tạp là $O(qn \log n)$. Chú ý phép chặt nhị phân trong giai đoạn ba có thể trực tiếp chặt trên miền giá trị, hoặc dùng tính chất nó là hàm từng đoạn để chặt trên các mực nước trong đoạn. Nếu dùng cách trước, $O(\log n)$ trong phân tích độ phức tạp có thể phải đổi thành $O(\text{số lần chặt miền giá trị})$. Để tránh sai số độ chính xác, số lần chặt này cần đủ lớn (khoảng 60). Các phân tích độ phức tạp của những thuật toán sau cũng có vấn đề tương tự.

Điểm kỳ vọng: có thể qua bốn subtask đầu, kỳ vọng 31 điểm.

2.4. Thuật toán bốn

Phân tích chất đặc biệt thỏa rằng ban đầu chỉ bể thứ nhất có nước. Ta thấy phần subtask này có một tính chất đẹp: mảng a không tăng, tức với mọi $1 \leq i < n$ có $a_i \geq a_{i+1}$. Kết luận này có thể suy ra từ mô hình ban đầu: khi điều chỉnh hai bể kề nhau, quan hệ thứ tự tương đối của chúng không thay đổi, nên nếu ban đầu thỏa tính chất này thì về sau chắc chắn vẫn thỏa.

Ta còn thấy đoạn có nước nhất định chỉ gồm K bể đầu, và với mọi $1 \leq i < K$, chắc chắn có $a_i \geq b_i$. Sau khi phân tích bình tĩnh, ba giai đoạn trước có thể rút gọn thành:

- Giai đoạn một: tìm đầu trái l của đoạn liên thông. Tính $V = \text{SUM}(l, x) - FL(l, x, h)$. Thực hiện thao tác $ML(l, x, h)$.
- Giai đoạn hai: tìm h lớn nhất sao cho $FL(l, r, h) - \text{SUM}(l, r) \leq V'$. Thực hiện thao tác $ML(l, r, h)$. Nếu h lớn hơn b_K , tăng K lên một, rồi sửa b_K , lặp lại quá trình này.

Vì giai đoạn hai ở đây thỏa mảng a giảm, và với mọi $l \leq i < r$ chắc chắn có $a_i \geq b_i$ (thật ra tính chất này cũng tồn tại trong giai đoạn ba ở trên), nên FL ở đây có thể tính trực tiếp bằng chặt nhị phân, tức tính tổng các giá trị lớn hơn h và số lượng giá trị nhỏ hơn h .

Ở đây còn có một cách đơn giản hơn. Vì tổng thể tích nước V cố định, ta có thể không duy trì mực nước thật. Ta chỉ cần biết bể có nước ngoài cùng bên phải là bể nào và mực nước của bể ấy, từ đó suy ra mực nước của mọi bể. Ta xét chỉ duy trì chiều cao vách ngăn, khi truy vấn thì tìm bể có nước ngoài cùng bên phải r và mực nước h của bể ấy. Do tính đơn điệu đã nhắc ở trên, ta chỉ cần tìm:

- tìm r lớn nhất, rồi tìm h lớn nhất, sao cho $FL(1, r, h) \leq V$.

Như vậy ta chỉ cần hỗ trợ thao tác FL . Tuy nhiên bên dưới vẫn giới thiệu một cách gần với lời giải chuẩn, tức cách duy trì FL, FR, ML, MR .

Ta thấy cần duy trì thông tin đoạn và thao tác đoạn, nên không khó nghĩ tới cây đoạn.

Trước hết cây đoạn chắc chắn cần duy trì tổng mực nước trong đoạn. Ta còn cần hỗ trợ truy vấn nhanh hai hàm FL và FR của đoạn với h bất kỳ, và hỗ trợ các phép sửa ML và MR với h bất kỳ; cây đoạn truyền thống không giải quyết trực tiếp được.

Xét một nút cây đoạn $[l, r]$ duy trì thông tin các bể trong $[l, r]$, cùng thông tin $r - l$ vách ngăn ở giữa. Giả sử các nút con đã có thể duy trì mực nước SUM, vách ngăn lớn nhất, và các thao tác FL, FR . Đặt

$$m = \left\lfloor \frac{l + r}{2} \right\rfloor,$$

khi đó hai nút con của $[l, r]$ là $[l, m]$ và $[m + 1, r]$. Ký hiệu

$$\text{HMAX}(l, r) = \max_{i=l}^{r-1} b_i.$$

Ta thấy

$$\begin{aligned} FL(l, r, h) &= FL(m + 1, r, h) \\ &\quad + FL(l, m, \max(h, \max(\text{HMAX}(m + 1, r), b_m))). \end{aligned}$$

Nếu cứ đệ quy trực tiếp thì là $O(r - l)$, nhưng nếu có thể chỉ chọn một phía để đệ quy thì sẽ đạt $O(\log(r - l))$.

Nếu $h \geq \text{HMAX}(m + 1, r)$, thì

$$FL(m + 1, r, h) = (r - m)h,$$

lúc này chỉ cần đệ quy tính phía còn lại. Nếu $h < \text{HMAX}(m + 1, r)$, chú ý rằng cần tính nhanh

$$FL(l, m, \max(h, \max(\text{HMAX}(m + 1, r), b_m))),$$

mà

$$\max(h, \max(\text{HMAX}(m + 1, r), b_m)) = \max(\text{HMAX}(m + 1, r), b_m),$$

đây là một hằng số không phụ thuộc vào h . Vì vậy ta cần ghi trong nút cây đoạn giá trị

$$FL(l, m, \max(\text{HMAX}(m + 1, r), b_m)).$$

Việc duy trì giá trị này cần gọi hàm FL của nút con; nhưng chỉ cần mọi nút trong cây con đều duy trì giá trị này, thì mọi hàm FL đều có thể tính trong $O(\log n)$.

Xét cách sửa. Chú ý rằng thao tác ML và MR về bản chất là thao tác sửa đoạn, nên các lazy tag không tương thích với nhau; có thể trực tiếp ghi đè, không cần xét gộp tag. Khi đánh tag ML hoặc MR trên cây đoạn, ta đồng thời ghi lại tham số h . Khi đánh tag, xét cách sửa nhanh thông tin đoạn; chú ý rằng cập nhật mực nước SUM chỉ cần truy vấn hàm FL hoặc FR . Loại tag này cũng rất dễ đẩy xuống.

Ta thu được một cây đoạn có thể hỗ trợ tổng mực nước, hỗ trợ các thao tác FL, FR, ML, MR . Vì hàm update (gộp thông tin hai con), hàm pushdown (đẩy tag xuống), cũng như các thao tác FL, FR, ML, MR trên một đoạn đơn đều là $O(\log n)$, nên độ phức tạp của thao tác trên đoạn bất kỳ là $O(\log^2 n)$.

Thuật toán bốn. Dùng phương pháp trên. Tính FL và sửa ML trong $O(\log^2 n)$. Cũng có thể trực tiếp dùng truy vấn FL để hỏi bể ngoài cùng bên phải và mực nước của bể ấy.

Độ phức tạp thuật toán: vì mỗi thao tác sửa cần chặt nhị phân, độ phức tạp là $O(q \log^3 n)$. Nếu trong giai đoạn hai đổi chặt nhị phân thành chặt ngay trên cây đoạn, có thể giảm độ phức tạp xuống $O(q \log^2 n)$.

Điểm kỳ vọng: có thể qua năm subtask đầu (cần chú ý hằng số), kỳ vọng 31 đến 61 điểm.

2.5. Thuật toán năm, thuật toán sáu

Thật ra cấu trúc dữ liệu trong thuật toán bốn đã có thể dùng để giải toàn bộ bài toán. Nhưng ta còn lại một vấn đề:

- Tìm đầu trái l của đoạn liên thông. Tìm r lớn nhất, rồi tìm h lớn nhất, sao cho $FL(l, r, h) \leq \text{SUM}(l, r)$.

Thuật toán năm. Dùng phương pháp trên. Tính FL, FR , sửa ML, MR trong $O(\log^2 n)$; thực hiện chặt nhị phân cộng truy vấn FL, FR trên cây đoạn trong $O(\log^3 n)$.

Độ phức tạp thuật toán: tổng độ phức tạp $O(q \log^3 n)$.

Điểm kỳ vọng: có thể qua mọi subtask (cần chú ý hằng số), kỳ vọng 31 đến 100 điểm.

Việc tìm h lớn nhất ở đây có thể làm trong $O(\log^2 n)$ bằng cách phỏng theo thuật toán trên. Mấu chốt là tìm r lớn nhất như thế nào. Chặt nhị phân trực tiếp là $O(\log^3 n)$. Ta thấy về bản chất nó yêu cầu nối một nút cây đoạn vào bên phải một đoạn bất kỳ, rồi truy vấn hàm FL hoặc FR của đoạn mới. Có thể xét trực tiếp tạo một nút cây đoạn mới để lưu thông tin đoạn mới này, chú ý mỗi nút cần lưu hai con trái phải của nó. Ta thấy sau khi gộp hai đoạn, truy vấn FL và FR vẫn là $O(\log n)$, do đó khi chặt trên cây đoạn, chỉ cần duy trì nút mới được gộp ra này là có thể giảm độ phức tạp xuống $O(\log^2 n)$.

Thuật toán sáu. Dùng phương pháp trên. Tính FL, FR , sửa ML, MR trong $O(\log^2 n)$; thực hiện chặt trên cây đoạn trong $O(\log^2 n)$.

Độ phức tạp thuật toán: tổng độ phức tạp $O(q \log^2 n)$.

Điểm kỳ vọng: có thể qua mọi subtask, kỳ vọng 100 điểm.

2.6. Các thuật toán khác

Do tính chất đặc biệt của bài này, còn một số thuật toán khác chưa được thảo luận; ở đây chỉ nhắc ngắn gọn.

Thuật toán bảy. Khi mô phỏng vết cạn, đẩy nước từng ô một và duy trì một hàng đợi đơn điệu.

Độ phức tạp thuật toán: tổng độ phức tạp $O(nq)$.

Điểm kỳ vọng: có thể qua bốn subtask đầu, kỳ vọng 31 điểm.

Thuật toán tám. Trong giai đoạn hai của ba giai đoạn ở trên, có thể trực tiếp sửa vết cạn; khi các giá trị trong đoạn của một nút cây đoạn là đơn điệu thì trực tiếp gán cả đoạn. Có thể chứng minh số nút được thăm là $O(\log n)$ trung bình, nhưng vì thao tác pushdown của cây đoạn cần $O(\log n)$, tổng độ phức tạp không đổi.

Độ phức tạp thuật toán: tổng độ phức tạp $O(q \log^2 n)$.

Điểm kỳ vọng: có thể qua mọi subtask, kỳ vọng 100 điểm.

3. Thiết lập dữ liệu kiểm thử

Bài này có tổng cộng 82 test. Có 11 loại dữ liệu ngẫu nhiên khác nhau, 3 loại dữ liệu thỏa tính chất đặc biệt, và 4 loại dữ liệu dựng tay. Trong đó dữ liệu dựng tay đồng thời kiểm tra độ phức tạp thời gian và độ chính xác của thuật toán. Đáp án cuối cùng được sinh bởi chương trình chuẩn dùng long double.

Ở đây chỉ nhắc tới một cách dựng dữ liệu có yêu cầu rất cao về độ chính xác.

Gọi m là một tham số (xấp xỉ một phần ba của n), đặt

$$S = \sum_{i=1}^m i.$$

Vì mọi số trong đầu vào của bài này sau khi nhân 10^7 đều là số nguyên, phần dưới trực tiếp dùng số nguyên để biểu thị giá trị đã nhân lên.

Chia toàn bộ dãy bể thành ba phần: phần thứ nhất cung cấp nước, phần thứ hai là một bậc thang, phần thứ ba nhận nước; mực nước trong phần thứ hai giảm dần. Mỗi phần có đúng m bể, và đặt đường chân trời là S .

Ban đầu mực nước và chiều cao vách ngăn của phần thứ nhất và phần thứ hai bằng nhau, phần thứ ba không có nước. Mỗi lần hạ chiều cao vách ngăn ngoài cùng bên phải của phần thứ nhất đúng S , rồi hạ vách ngăn ngoài cùng bên trái của phần thứ ba xuống S . Bảo đảm sau thao tác đầu tiên, S đơn vị thể tích nước này sẽ lấp bằng các bể của phần thứ hai; sau thao tác thứ hai, toàn bộ nước sẽ chảy vào dưới đường chân trời trong bể ngoài cùng bên trái của phần thứ ba, và không để lại hồ. Sau mỗi lần sửa, hỏi mực nước của bể ngoài cùng bên trái trong phần thứ ba.

Cuối cùng nói thêm vì sao bài này yêu cầu miền giá trị nhỏ hơn 100 và yêu cầu sai số tuyệt đối bảy chữ số. Trước hết, ta có thể thay đổi chiều cao vách ngăn của bể để thực hiện thao tác tương tự lấy phần thập phân trên mực nước, nên lúc này dùng sai số tuyệt đối hay sai số tương đối về bản chất là giống nhau. Mặt khác, vì lượng nước ban đầu không đổi, nếu có thuật toán duy trì các khối liên thông cùng mực nước thì khi miền giá trị nhỏ sẽ không thuận tiện để dựng dữ liệu có số lượng khối liên thông biến đổi lớn; vì vậy tác giả chọn dùng chín chữ số có nghĩa. Điều này có yêu cầu tương đối cao đối với độ chính xác của double; khi cài đặt nên cố gắng dùng phương pháp tính toán dấu phẩy động có sai số nhỏ.

4. Ước lượng độ khó

Subtask đầu tiên chỉ cần phân tích và mô phỏng đơn giản; dự kiến mọi thí sinh làm bài này đều có thể qua.

Subtask thứ hai cần thí sinh suy nghĩ một chút là có thể qua; dự kiến 80% thí sinh đều có thể qua.

Subtask thứ ba và thứ tư cần thí sinh tối ưu đơn giản thuật toán vét cạn, hoặc trực tiếp dùng một thuật toán vét cạn hiệu quả; dự kiến 60% thí sinh đều có thể qua.

Subtask thứ năm cần thí sinh phân tích và nghiên cứu bài toán sâu hơn, suy nghĩ kỹ rồi đưa ra cách làm; có độ khó tư duy nhất định. Độ khó code trung bình. Dự kiến 40% thí sinh đều có thể qua.

Subtask thứ sáu cần thí sinh tiếp tục phân tích, nghiên cứu và tối ưu bài toán; độ khó code cao. Dự kiến 20% thí sinh có thể qua.

Bài này ngoài độ khó tư duy và độ khó code, còn yêu cầu thí sinh phân tích và xử lý cẩn thận sai số độ chính xác của phép tính dấu phẩy động. Nếu thí sinh không phân tích kỹ thao tác, có thể còn đưa ra cách làm khá phức tạp. Nhìn chung độ khó của bài này khá cao.

5. Tổng kết

Cây đoạn được dùng cuối cùng trong bài này khác với cây đoạn thông thường. Về bản chất, mỗi nút của cây đoạn thông thường duy trì một tập hợp thông tin, và thông tin của mỗi nút có thể được lấy riêng ra. Cây đoạn trong bài này không chỉ duy trì một tập hợp thông tin, nó còn dựa vào cây nhị phân để hình thành một cấu trúc đệ quy; dù sửa hay hỏi một nút, đều phải dựa vào các nút con của nó. Khi truy vấn có

nhu cầu đặc biệt, ta còn có thể lấy ra một số nút để hình thành một cấu trúc đệ quy nằm ngoài cây nhị phân, điều này hơi giống tư tưởng của cây đoạn bền vững.

Kiến thức cần cho lời giải đúng của bài này chỉ liên quan tới cây đoạn, nhưng nó kiểm tra năng lực chuyển hóa mô hình, năng lực tư duy, năng lực phân tích vấn đề và năng lực code của thí sinh. Tuy đây là một bài cấu trúc dữ liệu, nó khác với bài cấu trúc dữ liệu truyền thống: không chỉ kiểm tra năng lực xử lý dữ liệu của thí sinh mà còn có yêu cầu cao về tư duy. Bài này không kiểm tra một cấu trúc dữ liệu mà thí sinh chưa nghiên cứu nghiêm túc hoặc chưa nắm chắc, mà kiểm tra cây đoạn, thứ thí sinh quen thuộc nhất, cơ bản nhất, và người thi NOIP đều biết dùng. Điều này cho thấy một bài hay không nhất thiết phải dựa vào cấu trúc dữ liệu hay kiến thức hóc lách, cũng không cần chỉ dựa vào lượng code khổng lồ để nâng ngưỡng. Người ra đề hy vọng bài này có thể đóng vai trò gợi mở, và mong được thấy mọi người nghĩ ra nhiều bài cấu trúc dữ liệu đa dạng hơn, thú vị hơn.

6. Lời cảm ơn

Xin cảm ơn Hội Máy tính Trung Quốc đã cung cấp nền tảng học tập và trao đổi.

Xin cảm ơn huấn luyện viên Zhang Ruizhe của đội tuyển tập huấn quốc gia đã hướng dẫn.

Xin cảm ơn thầy Li Shu của Trường Ngoại ngữ Nam Kinh đã quan tâm và hướng dẫn.

Xin cảm ơn bạn Wang Zeyuan đã thảo luận thuật toán với tôi và giúp kiểm tra đề.

Xin cảm ơn bạn Chen Sunli đã thẩm định bản thảo bài viết này.

Xin cảm ơn những bạn học đã đồng hành cùng tôi trong những năm qua.

Xin cảm ơn tất cả các thầy cô đã bồi dưỡng và hướng dẫn tôi.

Xin cảm ơn cha mẹ đã quan tâm và ủng hộ tôi.

Tài liệu tham khảo

1. CodeChef. Count on a Treap, problem code: COT5.
2. 2017 Multi-University Training Contest 5, problem 1005.